UME

Organization / Business Operating System

Comprehensive Design & Architecture Document

Document ID: UME-DA-002
Version: 1.0.0
Status: Draft — Comprehensive Baseline
Date: March 2026
Architecture Style: Kernel-Centric Modular Monorepo | Generated Domain Engines
Scope: Kernel + Devices/Drivers + 42 Organization Modules

Part I: System Overview & Design Principles

UME is a software-defined organization operating system (Business OS) built in Rust. It models a complete organization as a kernel-managed collection of domain subsystems — analogous to an operating system where kernel services manage business "hardware" (organizational functions), device drivers abstract external systems, and modules execute business logic.

Core Design Principles

Kernel-centric supervision — the UME kernel is the single composition root for all domain subsystems.
Dependency inversion — domain modules depend only on ume_core contracts; never on the kernel.
Fail-soft domain logic — missing or invalid domain records return structured insights, not panics.
Code generation as architecture — generated module fleets scaffold domain breadth rapidly.
Observability by default — every kernel service and org module emits structured events and metrics.
RBAC at every boundary — authorization is enforced on all mutating kernel facade operations.
Device/driver abstraction — all external system integrations are mediated by kernel-registered drivers.
Repository trait separation — storage backends are swappable without changing kernel or module APIs.

1.1 Architecture Overview

Layer	Components
Runtime Entrypoints	ume_runtime (CLI/REPL), ume_server (HTTP), clients/java (GUI + shell)
Kernel Layer	UmeKernel — ModuleRegistry, DeviceBus, ExecutorPool, EventBus, RBAC, Supervision, Orchestration
Kernel Services	MemoryManager, StorageManager, NetworkManager, LogAuditManager, TemplateEngine
Shared Core	ume_core — lifecycle, errors, IDs, time, metrics, contracts prelude
Domain Subsystems	chombo (Legal), soko (Marketing), shauri_os (320 firm modules)
Organization Modules	42 domain modules covering the complete enterprise function surface
Device Drivers	Storage, Identity, Communications, Event Broker, Finance, Document, HR, IoT, Analytics, Security
Org CPUs	ExecutorPool — named thread pools: kernel.critical, kernel.events, org.finance, org.hr, etc.

1.2 C4 System Context

```
┌──────────────────────────────────────────────────────────┐
│            Operators / External Systems                  │
│     Java Clients | HTTP APIs | IoT | ERP | HRIS          │
└──────────────────────────────────────────────────────────┘
                          │
                          │
          ┌──────────────────────────────────┐
          │       Runtime Entrypoints        │
          │  ume_runtime        ume_server   │
          └──────────────────────────────────┘
                          │
                          │
          ┌──────────────▼───────────────────┐
          │          UME KERNEL              │
          │  Module Registry | Event Bus     │
          │  RBAC | Supervisor | Executor    │
          │  Memory | Storage | Network      │
          │  Log/Audit | Template Engine     │
          └──────────────────────────────────┘
             │                          │
             │                          │
  ┌──────────▼──────────┐    ┌──────────▼──────────┐
  │  Domain Subsystems  │    │  42 Org Modules     │
  │  chombo | soko      │    │  Admin to Work Mgmt │
  │  shauri_os (320)    │    │  + Custom Modules   │
  └─────────────────────┘    └─────────────────────┘
             │                          │
             └──────────────────────────┘
                          │
                          │
          ┌──────────────▼───────────────────┐
          │    ume_core (shared contracts)   │
          │  lifecycle | errors | ids | time │
          └──────────────────────────────────┘
```

1.3 Module Index

#	Module Name	Domain
01	Organization Administration	Administration
02	Organization Analytics	Analytics
03	Backup, Recovery & State Management	Resilience
04	Board Management	Governance
05	Business Development & Analysis	Growth
06	Enterprise Content Management System	Content
07	Organization Communications	Communications
08	Client & Stakeholder Relationship Management	Relationships
09	Organization Design System	Design
10	Engineering & Technology	Engineering
11	ESG, CSR & Sustainability	Sustainability
12	Enterprise Engineering, Management & Administration	Enterprise Management
13	Legal Entity Management (Chombo)	Legal

14	Finance & Accounting	Finance
15	Governance, Risk & Compliance (GRC)	GRC
16	Human Resources	People
17	Investment Management	Investment
18	IT & Asset Management	IT
19	Enterprise Knowledge Management System	Knowledge
20	Learning & Development Management	Learning
21	Management & Strategy	Strategy
22	Marketing System (Soko)	Marketing
23	Master Data Management (MDM)	Data
24	Office & Facility Management	Facilities
25	Operations Management	Operations
26	Portal, Hub & Dashboard	Portal
27	Portfolio & Program Management	Portfolio
28	PR & Branding System	Brand
29	Process, Orchestration & Workflow	Process
30	Product, Services & Solution Management	Product
31	Production, Manufacturing & Fabrication	Production
32	Requirements Management	Requirements
33	Enterprise Risk Management	Risk
34	Sales Management	Sales
35	Enterprise Schedule Management	Schedule
36	Security, Privacy & Protection	Security
37	Logistics, Supply Chain & Warehouse	Supply Chain
38	Team & Cooperative Management	Teams
39	Organization Templating System	Templates
40	Enterprise Work Management	Work
41	Custom UME Modules	Extension
42	Custom Organization Modules	Extension

Part II: UME Kernel Architecture

The UME Kernel is the central supervisor, resource manager, and composition root for the entire organization operating system. Every subsystem, module, device driver, and executor operates under kernel authority. The kernel provides shared services — event routing, RBAC, orchestration, lifecycle supervision, memory management, storage, networking, and audit — so that organization modules can focus exclusively on their domain logic.

2.1 Kernel Core

The kernel core (UmeKernel struct) is the singleton composition root instantiated once at boot. It owns all subsystem handles, service registries, and configuration.

```
UmeKernel {
  config:         KernelConfig
  registry:       ModuleRegistry
  device_bus:     DeviceBus
  executor_pool:  ExecutorPool
  event_bus:      InMemoryEventBus
  rbac:           RbacEngine
  memory:         MemoryManager
  storage:        StorageManager
  network:        NetworkManager
  log_audit:      LogAuditManager
  supervisor:     SupervisorEngine
  orchestrator:   OrchestratorEngine
  // subsystem handles (Arc<RwLock<...>>)
  subsystems:     HashMap<DomainArea, Arc<dyn KernelSubsystem>>
}
```

KernelConfig Field	Description / Default
org_id	Canonical org identifier — org-{type}-{region}
org_name	Human-readable org display name
mode	Runtime mode: development | staging | production
enable_audit	Emit structured audit events — default true
enable_metrics	Collect performance metrics — default true
enable_rbac	Enforce RBAC on all facade calls — default true
max_module_restarts	Supervisor max restart attempts — default 3
backoff_ms	Supervisor restart backoff in ms — default 1000
storage_backend	Persistence adapter: memory | sqlite | postgres
event_backend	Event transport: memory | nats | kafka
log_level	Kernel log level: trace | debug | info | warn | error

Kernel Boot Sequence

1. Parse and validate KernelConfig from file / env / defaults.

2. Initialize LogAuditManager — all subsequent boot steps are logged.
3. Initialize MemoryManager — set up region allocators and cache layers.
4. Initialize StorageManager — open configured backend adapters.
5. Initialize NetworkManager — bind internal event transport channels.
6. Initialize RbacEngine — load role/permission definitions.
7. Initialize DeviceBus — enumerate and register organization device drivers.
8. Initialize ExecutorPool — spawn CPU/executor threads per configuration.
9. Initialize InMemoryEventBus (or configured broker adapter).
10. Initialize SupervisorEngine and OrchestratorEngine.
11. Load ModuleRegistry — instantiate and register all enabled org modules.
12. Run bootstrap_org() — generate org_id, activate default roles and modules.
13. Emit kernel.boot event; transition all subsystems to Running state.
14. Start HTTP server (ume_server) and expose operational endpoints.

2.2 Kernel Devices & Organization Device Drivers

UME uses a device/driver abstraction to model the integration points between the kernel and external organizational systems, data sources, communication channels, and hardware infrastructure. Each "device" represents a category of external resource; each "driver" is the adapter that implements the kernel DeviceDriver trait for a specific technology.

Concept: Devices as Org Resources

Just as a traditional OS kernel abstracts hardware (disk, NIC, GPU) through device drivers, the UME kernel abstracts organizational resources (ERP systems, identity providers, financial data feeds, IoT sensors, communication platforms) through org device drivers. All kernel modules interact with external resources exclusively through registered drivers.

```
pub trait DeviceDriver: Send + Sync {
    fn device_id(&self) -> &'static str;
    fn device_class(&self) -> DeviceClass;
    fn status(&self) -> DeviceStatus;
    fn initialize(&mut self, cfg: DeviceConfig) -> UmeResult<()>;
    fn shutdown(&mut self) -> UmeResult<()>;
    fn health_check(&self) -> DeviceHealthReport;
}
```

Device Classes & Driver Catalog

Device Class	Example Technologies	Driver Interface
Storage / Persistence	PostgreSQL, SQLite, Redis, S3, FileSystem	StorageDriver: read/write/query/migrate
Identity & Auth	OAuth2, LDAP, SAML, JWT issuer	IdentityDriver: authenticate/authorize/refresh
Communication	SMTP, SMS, Slack, Teams, WebSocket	CommDriver: send/receive/subscribe
Event Broker	NATS, Kafka, RabbitMQ, AMQP	BrokerDriver: publish/subscribe/ack

Financial Data	Banking APIs, ERP, Accounting systems	FinanceDriver: transact/reconcile/report
Document / Content	SharePoint, S3, CMS, Document stores	ContentDriver: create/read/version/search
Human Capital	HRIS, payroll, biometrics, badge systems	HrisDriver: sync/evaluate/report
IoT / Sensors	Environmental, manufacturing, location	SensorDriver: read/stream/alert
Analytics / BI	Data warehouses, BI tools, ML pipelines	AnalyticsDriver: query/stream/ingest
Security	SIEM, firewall, DLP, vault, HSM	SecurityDriver: scan/enforce/audit
Supply Chain	ERP, WMS, TMS, procurement platforms	SupplyDriver: track/order/reconcile
Calendar / Schedule	Exchange, Google Calendar, Gantt tools	ScheduleDriver: read/write/sync

Device Bus

The DeviceBus manages the lifecycle of all registered drivers. It provides discovery, health monitoring, and hot-swap semantics for driver replacement without kernel restart.

DeviceBus API	Behavior
register_driver(driver)	Add driver to bus; initialize and run health check
get_driver(device_id)	Retrieve a live driver handle by ID
list_drivers(class?)	List all registered drivers; optional class filter
health_check_all()	Run health check on every registered driver
replace_driver(id, new_driver)	Hot-swap a driver without module restart (graceful drain)
shutdown_all()	Gracefully shutdown all drivers in dependency order

2.3 Kernel Services

Kernel services are cross-cutting capabilities provided to all organization modules. Modules access services through the kernel service registry — they never instantiate services directly.

Service Registry

Kernel Service	Responsibility
ModuleRegistry	Register, lookup, list, and lifecycle-manage org modules
RbacEngine	Role, subject, and permission management; can() authorization decisions
SupervisorEngine	Monitor module state; apply restart/backoff/alert policies

OrchestratorEngine	Define, validate, and execute multi-step domain workflows
TemplateEngine	Render organization document, notification, and report templates
SchemaRegistry	Store and validate domain data schemas; enforce contract versions
ConfigService	Layered configuration: defaults -> file -> env -> runtime overrides
MetricsCollector	Collect and aggregate MetricPoint observations from all modules
NotificationService	Route alerts, notifications, and escalations to correct channels
SearchService	Full-text and structured search across all org domain data

2.4 Kernel Resources

Kernel resources are finite assets managed centrally to prevent contention and ensure fair allocation across organization modules.

Resource Type	Management Model
Executor Threads	ExecutorPool: named thread pools per domain; bounded work queue per pool
Memory Regions	MemoryManager: region-tagged allocations; eviction policies per region
Storage Connections	ConnectionPool: max-size pool per backend; timeout and retry policies
Event Bus Capacity	Backpressure on publish; configurable queue depth per topic
Network Sockets	Socket pool managed by NetworkManager; connection reuse and keepalive
Crypto / Key Material	Managed by SecurityDriver via kernel vault reference; no raw key exposure

2.5 Organization CPUs & Executors

The UME ExecutorPool abstracts computational resources for organization modules. Rather than raw OS threads, modules submit work to named executors — logical CPUs dedicated to domain execution priorities.

```
pub struct ExecutorPool {
    executors: HashMap<ExecutorId, Executor>,
    default_executor: ExecutorId,
}
```

```
pub struct Executor {
    id:             ExecutorId,
    domain:         DomainArea,
    thread_count:   usize,
    queue_depth:    usize,
    priority:       ExecutorPriority,   // Critical | High | Normal | Background
    work_queue:     Arc<Mutex<VecDeque<WorkItem>>>,
}
```

Built-in Executor Roles

Executor ID	Assigned Domain / Usage
kernel.critical	Kernel lifecycle, RBAC enforcement, supervisor decisions
kernel.events	Event bus dispatch and subscription fan-out
kernel.audit	Log and audit record persistence
org.finance	Finance, accounting, investment evaluation workloads
org.legal	Chombo compliance evaluation, filing plan synthesis
org.marketing	Soko strategy evaluation, signal processing
org.hr	HR evaluation, payroll batch, review processing
org.analytics	Analytics queries, report generation, BI workloads
org.ops	Operations, supply chain, production workloads
org.background	Non-critical background tasks: cleanup, sync, batch

2.6 Memory Management

The MemoryManager controls in-process heap allocation across kernel and all organization modules. It provides region-tagged allocation, cache management, and observability over per-domain memory utilization.

Memory Regions

Region	Purpose & Eviction Policy
kernel.hot	Kernel metadata, module registry — no eviction; always resident
domain.entities	Active domain aggregate records — LRU eviction when region exceeds threshold
domain.evaluations	Evaluation result cache — TTL-based expiry (configurable per module)
domain.signals	Ingested signal telemetry — sliding window eviction (configurable depth)
event.queue	In-flight event payloads — bounded FIFO; backpressure on overflow

audit.buffer	Audit record write buffer — flush on interval or threshold, then release
template.cache	Rendered template cache — LRU; invalidated on template version change
search.index	In-memory search index segments — rebuilt on restart if non-durable

Memory Manager API

```
allocate(region: RegionId, size: usize, tag: &str) -> AllocationHandle
release(handle: AllocationHandle)
region_utilization(region: RegionId) -> RegionStats
set_region_limit(region: RegionId, max_bytes: usize)
evict_region(region: RegionId, policy: EvictionPolicy)
snapshot_utilization() -> MemorySnapshot
```

2.7 File, Storage & Cache Management

The StorageManager provides a unified abstraction over all persistence backends. Modules never access storage directly — they call StorageManager APIs, which route to the appropriate backend driver based on data classification and configuration.

Storage Layers

Layer	Backend Options	Characteristics
Hot cache	In-memory HashMap/LRU	Sub-millisecond; non-durable; bounded by MemoryManager region limits
Warm store	SQLite, embedded RocksDB	Low-latency; single-node durable; suitable for development/small deployments
Cold store	PostgreSQL, MySQL, CockroachDB	Full ACID; multi-tenant capable; production tier
Object store	S3, Azure Blob, MinIO, filesystem	Large artifacts, documents, backups, media
Search index	MeiliSearch, Tantivy, Elasticsearch	Full-text and structured search queries
Cache layer	Redis, Memcached, Valkey	Shared cross-process cache; distributed invalidation

StorageManager API

```
get<T>(region, key) -> UmeResult<Option<T>>
put<T>(region, key, value, ttl?) -> UmeResult<()>
delete(region, key) -> UmeResult<bool>
query<T>(region, filter: QueryFilter) -> UmeResult<Vec<T>>
begin_transaction() -> TransactionHandle
commit(tx: TransactionHandle) -> UmeResult<()>
rollback(tx: TransactionHandle)
migrate(migration: SchemaMigration) -> UmeResult<()>
```

```
backup(scope: BackupScope, dest: &str) -> UmeResult<BackupManifest>
restore(manifest: BackupManifest, source: &str) -> UmeResult<()>
```

2.8 Network, Communications & Event Management

The NetworkManager and InMemoryEventBus (or broker adapter) together handle all inter-module communication, external integration messaging, and real-time event streaming.

Event Bus Architecture

Concept	Detail
Topic namespace	Hierarchical: {domain}.{resource}.{verb} — e.g. org.finance.invoice.created
Publisher API	publish(event: KernelEvent) — fire-and-forget; backpressure on queue saturation
Subscriber API	subscribe(topic_pattern, handler) — glob patterns supported
Delivery semantics	At-least-once with deduplication key in current revision
Persistence	Configurable: ephemeral | append-log | broker-backed
Replay	replay_from(topic, cursor: EventCursor) — for audit and recovery scenarios
Dead letter queue	Failed deliveries routed to dlq.{topic} for inspection and replay

Communication Channels

Channel Type	Used By
Internal event bus	All kernel services and org modules for intra-process messaging
HTTP REST	ume_server external API; Java and generic clients
WebSocket	Real-time push to portal/dashboard module and external UIs
gRPC (future)	High-throughput inter-service calls in distributed deployment
Email / SMS / Push	Notification service routing via CommDriver adapters
Webhook	Outbound event delivery to external subscribers (CRM, ERP integrations)

2.9 Log & Audit Management

The LogAuditManager provides structured logging, immutable audit recording, and compliance-grade event trails for all kernel and organization module operations.

Log Levels & Structured Format

Log Level	Usage
TRACE	Verbose per-operation debug data; kernel internals; never in production
DEBUG	Module-level diagnostic output; evaluation internals; development only
INFO	Normal operational events: module start/stop, evaluations, user actions
WARN	Recoverable anomalies: slow evaluation, cache miss spikes, soft errors
ERROR	Non-fatal failures: operation failed but system continues
CRITICAL	Fatal conditions: kernel fault, data corruption, security breach

Audit Record Structure

```
AuditRecord {
    id:           String,      // UUID v4
    at:           String,      // UTC RFC3339
    actor:        String,      // subject_id performing action
    action:       String,      // {domain}.{resource}.{verb}
    resource_id:  String,      // affected entity/resource ID
    outcome:      AuditOutcome,// Success | Failure | Denied
    context:      JsonValue,   // additional structured context
    session_id:   String,      // correlates related actions
    ip_address:   Option<String>,
    user_agent:   Option<String>,
}
```

Audit Storage & Compliance

- Audit records are written to an append-only audit log — records are immutable after write.
- Audit log is replicated to the configured StorageManager cold-store backend for durability.
- Audit queries: filter by actor, action, resource, time range, outcome.
- Retention policy is configurable per compliance requirement (SOC2, ISO27001, GDPR).
- Audit events are also published to the event bus (topic: audit.record.created) for SIEM integration.
- RBAC denial events always produce audit records regardless of module-level audit settings.

2.10 Module Registry & Lifecycle Management

The ModuleRegistry is the master catalog of all installed organization modules. It enforces unique module IDs, manages lifecycle state transitions, and serves as the query interface for module discovery.

```
pub trait KernelModule: Send + Sync {
```

```
    fn id(&self)            -> &'static str;
    fn name(&self)          -> &'static str;
    fn domain(&self)        -> DomainArea;
    fn version(&self)       -> &'static str;
    fn state(&self)         -> LifecycleState;
    fn start(&self)         -> UmeResult<()>;
    fn stop(&self)          -> UmeResult<()>;
    fn pause(&self)         -> UmeResult<()>;
    fn resume(&self)        -> UmeResult<()>;
    fn health_check(&self) -> ModuleHealthReport;
    fn describe(&self)      -> ModuleDescriptor;
    fn dependencies(&self) -> Vec<&'static str>;
}
```

Lifecycle State Machine

From State	Allowed Transitions
Bootstrapping	-> Ready (success) | -> Error (failure)
Ready	-> Running (start called) | -> Stopped (manual)
Running	-> Degraded (soft error) | -> Paused (pause called) | -> Stopped (stop called) | -> Error (fatal)
Degraded	-> Running (recovered) | -> Error (worsened) | -> Stopped (manual)
Paused	-> Running (resume called) | -> Stopped (stop called)
Stopped	-> Ready (re-init) | [terminal unless explicitly restarted]
Error	-> Ready (restart by supervisor) | -> Stopped (max restarts exceeded)

Part III: UME Organization Modules

Organization modules are the domain engines of the UME Business OS. Each module is a first-class kernel-managed subsystem implementing the KernelModule trait. Modules are independently versioned, supervised, and composable. They expose structured APIs to the kernel facade, emit domain events to the event bus, and enforce RBAC on all mutating operations.

Module Contract Summary

Every org module MUST implement: id, name, domain, version, state, start, stop, pause, resume, health_check, describe, dependencies.
Every org module MUST: enforce RBAC on mutating operations; emit domain events; write audit records for all state-changing calls; expose a structured ModuleDescriptor.
Every org module MUST NOT: depend on other org modules directly — only on ume_core contracts and kernel service facades.

Module 01: Organization Administration

Domain: Administration | ume.domain.administration | Module ID: ume.module.01

Purpose & Responsibility

Central administrative coordination module for the organization. Manages org structure records, policy registry, delegation authorities, internal notices, administrative workflows, and the master configuration of organizational units.

- Maintain the authoritative organization structure (entities, departments, divisions, SBUs)
- Manage administrative policy library: policies, procedures, standing orders
- Coordinate delegation of authority matrices and signatory registers
- Handle internal notice and circular distribution
- Administer meeting coordination and resolution tracking
- Manage statutory register and corporate secretarial records

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Org Structure Manager	Org chart & hierarchy	Maintain entities, departments, reporting lines, cost centers
Policy Registry	Policy lifecycle	Draft -> review -> approve -> publish -> retire policy lifecycle
Delegation Manager	Authority matrix	Delegation of authority levels, signatories, approvals limits
Notice Board	Internal communications	Circulars, notices, announcements with acknowledgement tracking
Admin Workflow	Process automation	Approval chains, form routing, escalation paths
Corporate Secretariat	Statutory records	Resolutions, registers, filings, governance records

Core Data Models

Model / Entity	Purpose & Key Fields
OrgUnit { id, name, type, parent_id, cost_center, status }	Organizational unit in the hierarchy
AdminPolicy { id, title, domain, version, status, owner, effective_date, body }	Policy document record
DelegationMatrix { id, org_unit_id, authority_type, max_value_cents, approver_ids }	Authority delegation entry
InternalNotice { id, title, body, issued_by, issued_at,	Internal communication

audience_groups, ack_required }	
AdminWorkflowInstance { id, workflow_id, initiator, current_step, status, created_at }	Active workflow instance
Resolution { id, body, resolved_by, resolved_at, binding, references }	Board or committee resolution

Key Operations & API Surface

```
register_org_unit(unit: OrgUnit) -> UmeResult<OrgUnit>
get_org_hierarchy(root_id: &str) -> OrgHierarchy
create_policy(policy: AdminPolicy) -> UmeResult<AdminPolicy>
publish_notice(notice: InternalNotice) -> UmeResult<()>
create_delegation(matrix: DelegationMatrix) -> UmeResult<()>
evaluate_authority(subject_id: &str, action: &str, value_cents: i64) -> AuthorityDecision
list_resolutions(filter: ResolutionFilter) -> Vec<Resolution>
```

Domain Events

Event Topic	Trigger Condition
admin.org_unit.created	New organizational unit registered in hierarchy
admin.policy.published	Policy moved to published/active state
admin.notice.issued	Internal notice or circular distributed
admin.delegation.updated	Delegation of authority matrix modified
admin.resolution.recorded	Board/committee resolution recorded

Integration Points

- HR Module — sync org unit structure with headcount and reporting lines
- Legal Entity Module — link corporate entities to org units
- GRC Module — policies feed compliance frameworks
- Work Management Module — admin workflows become work items
- Portal Module — notices and org chart surfaced in employee portal

Configuration

Config Key	Description / Default
admin.hierarchy.max_depth	Maximum org chart depth — default 10
admin.policy.approval_quorum	Required approvers for policy publication — default 2
admin.notice.default_audience	Default audience group for notices — default org.all

Module 02: Organization Analytics

Domain: Analytics | ume.domain.analytics | Module ID: ume.module.02

Purpose & Responsibility

Enterprise analytics and business intelligence engine. Aggregates data across all organization modules, computes KPIs, powers dashboards, enables ad-hoc query, predictive analytics, and report generation for decision-making at all organizational levels.

- Aggregate and normalize data streams from all active org modules
- Compute and maintain organizational KPIs and performance metrics
- Provide ad-hoc and scheduled report generation
- Power real-time and near-real-time dashboard data feeds
- Support predictive analytics and trend analysis models
- Manage the analytics data catalog and metric registry

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Metric Registry	KPI definitions	Define, version, and own all organizational KPI definitions
Data Aggregator	Cross-module ETL	Subscribe to domain events; build analytics data store
Query Engine	SQL / analytical queries	Execute OLAP-style queries over aggregated data
Report Builder	Report generation	Scheduled and on-demand report synthesis with template integration
Dashboard Feed	Real-time data	WebSocket feed powering portal dashboard widgets
Prediction Engine	ML-backed analytics	Trend forecasting, anomaly detection, scenario modeling
Data Catalog	Schema metadata	Track all available analytics datasets, fields, lineage

Core Data Models

Model / Entity	Purpose & Key Fields
MetricDefinition { id, name, formula, domain, unit, frequency, owner }	KPI specification
MetricObservation { metric_id, org_unit_id, period, value, computed_at }	KPI data point
AnalyticsReport { id, name, query, schedule, last_run_at, output_format }	Report definition

Dashboard { id, name, widgets: Vec<WidgetConfig>, audience, refresh_sec }	Dashboard configuration
PredictionModel { id, algorithm, training_dataset, accuracy, deployed_at }	Deployed prediction model
DataCatalogEntry { id, source_module, dataset_name, schema, lineage_ids }	Analytics dataset descriptor

Key Operations & API Surface

```
register_metric(def: MetricDefinition) -> UmeResult<()>
record_observation(obs: MetricObservation) -> UmeResult<()>
query(q: AnalyticsQuery) -> UmeResult<QueryResult>
run_report(report_id: &str) -> UmeResult<ReportOutput>
get_dashboard_feed(dashboard_id: &str) -> DashboardSnapshot
predict(model_id: &str, input: PredictionInput) -> PredictionOutput
search_catalog(terms: &str) -> Vec<DataCatalogEntry>
```

Domain Events

Event Topic	Trigger Condition
analytics.metric.recorded	New KPI observation persisted
analytics.report.completed	Scheduled or on-demand report generated
analytics.anomaly.detected	Prediction engine flagged anomaly in metric stream
analytics.dashboard.refreshed	Dashboard snapshot updated

Integration Points

- All org modules — subscribe to their domain events as analytics input
- Portal/Dashboard Module — serve dashboard snapshots and drill-down data
- Management & Strategy Module — KPI data feeds strategic review cycles
- Risk Management Module — anomaly signals feed risk indicators
- HR Module — people analytics and workforce metrics

Configuration

Config Key	Description / Default
analytics.refresh_interval_sec	Default metric aggregation interval — default 60
analytics.retention_days	Analytics data retention window — default 730
analytics.prediction.enabled	Enable ML prediction engine — default false (requires model deployment)

Module 03: Backup, Recovery & State Management

Domain: Resilience | ume.domain.backup | Module ID: ume.module.03

Purpose & Responsibility

Enterprise-grade backup, state snapshot, disaster recovery, and restore orchestration module. Ensures organizational data and system state can be captured, versioned, and restored to any consistent point in time with configurable RPO and RTO targets.

- Schedule and execute full, incremental, and differential backups across all storage regions
- Maintain backup manifests with integrity checksums and chain metadata
- Orchestrate disaster recovery procedures including cross-region failover
- Manage system state snapshots for kernel, modules, and domain data
- Coordinate restore operations with validation and rollback capability
- Test backup integrity through automated restore verification

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Backup Scheduler	Backup jobs	Cron-based and event-triggered backup job scheduling
Snapshot Engine	State capture	Consistent point-in-time snapshots of kernel and domain state
Integrity Verifier	Checksum & verify	SHA-256 checksum all backup artifacts; automated verification runs
Recovery Orchestrator	DR procedures	Multi-step recovery playbooks with dependency ordering
Restore Engine	Data restoration	Selective and full restore with pre/post validation hooks
DR Test Runner	Restore verification	Automated restore-to-sandbox verification on schedule
Manifest Registry	Backup inventory	Catalog of all backup artifacts with chain and lifecycle metadata

Core Data Models

Model / Entity	Purpose & Key Fields
BackupJob { id, scope, type, schedule, retention_days, dest_driver_id, last_run }	Backup job definition
BackupManifest { id, job_id, created_at, checksum, chain_id, artifact_path, size_bytes }	Backup artifact record
StateSnapshot { id, module_id, captured_at,	Module state snapshot

state_hash, artifact_path, verified }	
RecoveryPlan { id, name, steps: Vec<RecoveryStep>, rpo_hours, rto_hours, tested_at }	DR plan definition
RestoreRequest { id, manifest_id, target_env, scope, initiated_by, status }	Active restore operation

Key Operations & API Surface

```
schedule_backup(job: BackupJob) -> UmeResult<BackupJob>
run_backup_now(job_id: &str) -> UmeResult<BackupManifest>
capture_snapshot(module_id: &str) -> UmeResult<StateSnapshot>
initiate_restore(request: RestoreRequest) -> UmeResult<RestoreHandle>
verify_backup(manifest_id: &str) -> UmeResult<IntegrityReport>
execute_recovery_plan(plan_id: &str, env: &str) -> UmeResult<RecoveryLog>
list_manifests(filter: ManifestFilter) -> Vec<BackupManifest>
```

Domain Events

Event Topic	Trigger Condition
backup.job.completed	Backup job finished — includes manifest ID and checksum
backup.integrity.failed	Checksum verification failure detected
backup.restore.initiated	Restore operation started
backup.restore.completed	Restore finished — includes verification status
backup.dr_test.passed	Automated DR test completed successfully
backup.dr_test.failed	Automated DR test failed — escalation triggered

Integration Points

- StorageManager — backup and restore routes through storage driver abstraction
- All org modules — capture_snapshot() called on each module during full backup
- Security Module — encryption of backup artifacts via SecurityDriver vault
- Audit Module — all restore and DR operations produce mandatory audit records
- Notification Service — backup failures and DR events trigger alerts

Configuration

Config Key	Description / Default
backup.default_retention_days	Default backup artifact retention — default 90
backup.snapshot_interval_hours	Module snapshot frequency — default 24
backup.verify_on_create	Run integrity check immediately after backup — default true
backup.dr_test_schedule	Cron expression for automated DR tests — default 0 2 1 * * (monthly)

Module 04: Board Management

Domain: Governance | ume.domain.board | Module ID: ume.module.04

Purpose & Responsibility

Digital governance platform for the board of directors and executive committees. Manages board composition, meeting lifecycle, agenda coordination, resolution tracking, director duties, committee management, and shareholder/stakeholder reporting.

- Maintain board and committee composition, tenure, and independence assessments
- Coordinate full meeting lifecycle: notice, agenda, materials, quorum, minutes, resolution
- Track director duties, declarations of interest, and fit-and-proper assessments
- Manage committee charters, committee minutes, and committee reporting
- Produce board packs and governance reports
- Track shareholder register interface and shareholder-related board decisions

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Board Composition Manager	Director registry	Track directors, independence status, terms, qualifications, declarations
Meeting Coordinator	Meeting lifecycle	Notice -> agenda -> materials distribution -> quorum check -> minutes -> resolution
Committee Manager	Committee governance	Committee charter, composition, quorum rules, minutes, reporting
Board Pack Builder	Document assembly	Automated assembly of board pack from module data and templates
Resolution Tracker	Decision registry	Formal resolutions with binding status, reference, and implementation tracking
Governance Reporter	Board reporting	Board effectiveness reports, compliance dashboards, shareholder updates

Core Data Models

Model / Entity	Purpose & Key Fields
Director { id, name, type, appointed_at, term_end, independence_status, declarations }	Board member record
BoardMeeting { id, type, scheduled_at, location, quorum_required, status, agenda_id }	Board meeting instance
Agenda { id, meeting_id, items: Vec<AgendaItem>, distributed_at, approved }	Meeting agenda

Resolution { id, meeting_id, text, proposer_id, seconder_id, vote_result, binding, ref_no }	Formal resolution
CommitteeCharter { id, name, purpose, composition_rules, reporting_to, quorum }	Committee governance document
DeclarationOfInterest { id, director_id, nature, declared_at, materiality, managed_by }	Director conflict declaration

Key Operations & API Surface

```
register_director(director: Director) -> UmeResult<Director>
schedule_meeting(meeting: BoardMeeting) -> UmeResult<BoardMeeting>
create_agenda(agenda: Agenda) -> UmeResult<Agenda>
record_resolution(resolution: Resolution) -> UmeResult<Resolution>
generate_board_pack(meeting_id: &str) -> UmeResult<BoardPack>
check_quorum(meeting_id: &str) -> QuorumStatus
list_open_resolutions(filter: ResolutionFilter) -> Vec<Resolution>
```

Domain Events

Event Topic	Trigger Condition
board.meeting.scheduled	New board meeting added to calendar
board.resolution.passed	Resolution formally passed and recorded
board.quorum.failed	Meeting quorum not achieved — meeting lapsed
board.declaration.filed	Director declaration of interest recorded
board.pack.generated	Board pack documents assembled and distributed

Integration Points

- Legal Entity Module — board decisions affect corporate structure and statutory filings
- GRC Module — board resolutions feed governance and compliance records
- Finance Module — board approvals gate capital and budget decisions
- Administration Module — resolutions populate statutory register
- Schedule Management Module — board meetings appear in organizational calendar

Configuration

Config Key	Description / Default
board.quorum.default_percent	Default quorum threshold percentage — default 50
board.pack.lead_days	Days before meeting to distribute board pack — default 5
board.resolution.binding_requires_unanimous	Require unanimity for binding resolutions — default false

Module 05: Business Development & Analysis

Domain: Growth | ume.domain.bizdev | Module ID: ume.module.05

Purpose & Responsibility

Organizational growth and opportunity development module. Manages the pipeline of new business opportunities, partnership development, market analysis, competitive intelligence, deal structuring, and business case development from initial identification through to strategic execution.

- Manage the full business development pipeline from prospect to closed opportunity
- Track partnerships, alliances, and channel development
- Coordinate business case and feasibility study development
- Maintain competitive intelligence and market landscape analysis
- Support M&A target identification, screening, and due diligence coordination
- Link BD outcomes to strategic planning and portfolio management

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Opportunity Pipeline	Deal tracking	Stage-gated opportunity management from lead to close
Partnership Manager	Alliance management	Partnership agreements, SLAs, co-development tracks
Market Intelligence	Competitive analysis	Market landscape, competitor profiles, win/loss analysis
Business Case Builder	Feasibility & ROI	Structured business case templates with financial modeling hooks
M&A Tracker	Acquisition pipeline	Target screening, due diligence coordination, integration planning
BD Analytics	Performance metrics	Pipeline conversion, revenue forecasting, BD KPI tracking

Core Data Models

Model / Entity	Purpose & Key Fields
BizDevOpportunity { id, name, type, stage, value_cents, probability, owner_id, close_date }	BD pipeline opportunity
Partnership { id, partner_org, type, status, agreement_id, review_date, kpis }	Partnership record
MarketIntelligence { id, topic, source, insight, confidence, captured_at }	Market intelligence entry

BusinessCase { id, opportunity_id, summary, costs_cents, benefits_cents, npv_cents, status }	Business case document
MATarget { id, target_name, strategic_fit_score, valuation_cents, dd_status, integration_plan_id }	M&A target record

Key Operations & API Surface

```
create_opportunity(opp: BizDevOpportunity) -> UmeResult<BizDevOpportunity>
advance_stage(opp_id: &str, stage: OpportunityStage) -> UmeResult<()>
register_partnership(p: Partnership) -> UmeResult<Partnership>
add_market_intelligence(intel: MarketIntelligence) -> UmeResult<()>
create_business_case(bc: BusinessCase) -> UmeResult<BusinessCase>
pipeline_summary() -> BDPipelineSummary
screen_ma_target(target: MATarget) -> UmeResult<MAScreeningReport>
```

Domain Events

Event Topic	Trigger Condition
bizdev.opportunity.stage_changed	Opportunity advanced or retreated in pipeline stages
bizdev.opportunity.closed_won	Opportunity successfully closed
bizdev.partnership.activated	Partnership agreement moved to active status
bizdev.ma_target.approved	M&A target approved for due diligence
bizdev.business_case.approved	Business case approved for execution

Integration Points

- Management & Strategy Module — BD pipeline feeds strategic objectives and OKRs
- Finance Module — business case financial models link to budget and investment planning
- Legal Entity Module — partnership agreements and M&A trigger legal entity changes
- Portfolio Management Module — approved BD opportunities become portfolio items
- Sales Management Module — closed BD opportunities hand off to sales for execution

Configuration

Config Key	Description / Default
bizdev.pipeline.stages	Ordered stage definitions — default: Prospect,Qualify,Propose,Negotiate,Close
bizdev.opportunity.stale_days	Days before opportunity flagged as stale — default 30

Module 06: Enterprise Content Management System

Domain: Content | ume.domain.cms | Module ID: ume.module.06

Purpose & Responsibility

Centralized enterprise content management module handling the full lifecycle of organizational content assets — documents, policies, knowledge articles, media, and structured content — from creation through review, publication, archival, and disposal.

- Manage content asset lifecycle: draft -> review -> approve -> publish -> archive -> dispose
- Provide structured content types with configurable metadata schemas
- Maintain content version history with diff and rollback capability
- Enforce retention policies and disposition schedules
- Power search and discovery across all organizational content
- Manage permissions and access controls on content items

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Content Repository	Asset storage	Store, retrieve, and version content assets via StorageDriver
Workflow Engine	Content lifecycle	Configurable approval and review workflows per content type
Metadata Manager	Schema & taxonomy	Configurable metadata schemas; taxonomy and tag management
Version Control	Diff and history	Git-style versioning with diff, blame, and rollback
Retention Engine	Policy enforcement	Automated retention schedule enforcement and legal hold management
Content Search	Discovery	Full-text and metadata search powered by SearchService
Access Control	Permissions	Content-level permission rules integrated with RbacEngine

Core Data Models

Model / Entity	Purpose & Key Fields
ContentAsset { id, type, title, status, owner_id, content_driver_ref, version, metadata }	Content asset master record
ContentVersion { id, asset_id, version_no, created_by, created_at, change_summary, artifact_ref }	Version snapshot

ContentType { id, name, schema: JsonSchema, workflow_id, retention_years }	Content type definition
RetentionPolicy { id, content_type_id, retain_years, trigger, action: Archive | Delete | Review }	Retention rule
LegalHold { id, asset_ids, reason, placed_by, placed_at, lifted_at }	Legal hold override on retention

Key Operations & API Surface

```
create_content(asset: ContentAsset) -> UmeResult<ContentAsset>
publish_content(asset_id: &str) -> UmeResult<()>
update_content(asset_id: &str, update: ContentUpdate) -> UmeResult<ContentVersion>
search_content(query: ContentQuery) -> Vec<ContentSearchResult>
apply_retention_policy(policy_id: &str) -> RetentionReport
place_legal_hold(hold: LegalHold) -> UmeResult<()>
get_version_history(asset_id: &str) -> Vec<ContentVersion>
```

Domain Events

Event Topic	Trigger Condition
cms.content.published	Content asset moved to published state
cms.content.archived	Content asset archived per retention schedule
cms.legal_hold.placed	Legal hold applied to one or more content assets
cms.version.created	New content version snapshot created
cms.retention.enforced	Retention policy disposition action executed

Integration Points

- Administration Module — policies stored as CMS content assets
- Knowledge Management Module — knowledge articles are CMS content types
- HR Module — employee documents, contracts stored via CMS
- Legal Entity Module — legal documents stored and versioned via CMS
- Backup Module — CMS artifact store included in backup scope

Configuration

Config Key	Description / Default
cms.max_asset_size_mb	Maximum individual asset size in MB — default 100
cms.default_retention_years	Default retention if no policy configured — default 7
cms.version_history_limit	Max versions retained per asset — default 50

Module 07: Organization Communications

Domain: Communications | ume.domain.communications | Module ID: ume.module.07

Purpose & Responsibility

Unified internal and external communication management module. Orchestrates messaging across all communication channels, manages communication campaigns, crisis communications, stakeholder messaging, and provides a single source of truth for organizational communication records.

- Route and deliver messages across all configured communication channels (email, SMS, push, in-app)
- Manage communication templates and branded message libraries
- Coordinate internal broadcast and targeted communications
- Manage crisis and emergency communication playbooks
- Track communication delivery, open rates, and response metrics
- Archive all organizational communications for compliance and audit

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Message Router	Channel dispatch	Route messages to appropriate driver based on type and preference
Communication Templates	Template library	Branded templates with variable substitution; linked to Templating Module
Broadcast Manager	Mass communications	Targeted broadcast to org units, roles, tags, external lists
Crisis Comm Engine	Emergency comms	Pre-defined crisis playbooks with automated escalation sequences
Delivery Tracker	Metrics & status	Track send, delivered, opened, clicked, replied per message
Comm Archive	Compliance store	Immutable archive of all sent/received communications via StorageDriver
Preference Manager	Channel preferences	Per-user channel preference management; opt-out handling

Core Data Models

Model / Entity	Purpose & Key Fields
Message { id, type, subject, body, from, to, channel, template_id, sent_at, status }	Outbound message record
CommCampaign { id, name, audience, channel, template_id, schedule, status, metrics }	Communication campaign

CrisisPlaybook { id, name, trigger, escalation_steps, audience_groups, channel_priority }	Crisis communication plan
DeliveryRecord { message_id, recipient, channel, status, delivered_at, opened_at }	Delivery tracking entry
CommPreference { subject_id, channel_preferences, opt_outs, last_updated }	User channel preference profile

Key Operations & API Surface

```
send_message(message: Message) -> UmeResult<MessageHandle>
launch_campaign(campaign: CommCampaign) -> UmeResult<CampaignHandle>
activate_crisis_playbook(playbook_id: &str, context: JsonValue) ->
UmeResult<CrisisHandle>
get_delivery_report(campaign_id: &str) -> DeliveryReport
set_preference(pref: CommPreference) -> UmeResult<()>
search_archive(query: CommQuery) -> Vec<Message>
```

Domain Events

Event Topic	Trigger Condition
comms.message.sent	Message dispatched via channel driver
comms.message.delivered	Delivery confirmation received from channel
comms.crisis.activated	Crisis communication playbook triggered
comms.campaign.completed	Communication campaign dispatch finished
comms.opt_out.recorded	Recipient opted out of a communication channel

Integration Points

- All org modules — receive notification routing requests from any domain event
- HR Module — employee contact details and communication preferences
- Security Module — security alerts routed through crisis comm playbooks
- Portal Module — in-app notification channel served via portal
- CRM Module — external stakeholder communications tracked in CRM records

Configuration

Config Key	Description / Default
comms.default_channel	Default communication channel — default email
comms.archive_retention_days	Communication archive retention — default 2555 (7 years)
comms.rate_limit_per_min	Max outbound messages per minute — default 500

Module 08: Client & Stakeholder Relationship Management (CRM)

Domain: Relationships | ume.domain.crm | Module ID: ume.module.08

Purpose & Responsibility

Enterprise CRM module managing relationships with clients, customers, prospects, partners, suppliers, investors, regulators, and all external stakeholders. Provides a 360-degree view of every external relationship including interaction history, engagement scoring, contract status, and relationship health monitoring.

- Maintain a unified stakeholder registry with deduplication and master data governance
- Track all interactions, touchpoints, and relationship history per stakeholder
- Manage client accounts, contracts, and service agreements
- Score and segment stakeholders by relationship health, value, and risk
- Coordinate stakeholder engagement plans and contact scheduling
- Produce relationship analytics and key account management reports

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Stakeholder Registry	Contact master data	Unified profile with dedup, merge, and MDM integration
Interaction Tracker	History log	Log all calls, emails, meetings, tickets per stakeholder
Account Manager	Key accounts	Account plans, revenue tracking, satisfaction scores, red flags
Contract Repository	Agreement management	Contracts, MSAs, NDAs, SLAs linked to stakeholder profiles
Engagement Planner	Contact scheduling	Proactive engagement schedule with cadence rules and reminders
Relationship Scoring	Health metrics	Compute relationship health scores from interaction and outcome data
CRM Analytics	Pipeline & retention	Revenue by account, churn prediction, NPS, account health dashboard

Core Data Models

Model / Entity	Purpose & Key Fields
Stakeholder { id, name, type, org_name, email, phone, tier, owner_id, tags, health_score }	External stakeholder profile
Interaction { id, stakeholder_id, type, channel, summary, outcome, owner_id, occurred_at }	Interaction record

Account { id, stakeholder_id, revenue_cents, status, assigned_to, contract_ids, risk_flag }	Key account record
Contract { id, stakeholder_id, type, value_cents, start_date, end_date, status, signed_at }	Contract/agreement record
EngagementPlan { id, account_id, contacts: Vec<PlannedContact>, cadence, next_due }	Stakeholder engagement plan
RelationshipScore { stakeholder_id, score, drivers, computed_at, trend }	Computed relationship health score

Key Operations & API Surface

```
upsert_stakeholder(s: Stakeholder) -> UmeResult<Stakeholder>
log_interaction(i: Interaction) -> UmeResult<Interaction>
create_account(a: Account) -> UmeResult<Account>
upsert_contract(c: Contract) -> UmeResult<Contract>
compute_relationship_score(stakeholder_id: &str) -> RelationshipScore
list_accounts(filter: AccountFilter) -> Vec<Account>
crm_summary() -> CRMSummary
```

Domain Events

Event Topic	Trigger Condition
crm.stakeholder.created	New stakeholder profile registered
crm.interaction.logged	Interaction record added to stakeholder history
crm.contract.signed	Contract moved to signed/active status
crm.account.flagged	Key account flagged for risk or attention
crm.health_score.degraded	Relationship health score dropped below threshold

Integration Points

- Sales Module — CRM accounts and opportunities shared with sales pipeline
- Communications Module — outbound communications logged as CRM interactions
- Finance Module — contract values feed revenue and accounts receivable
- MDM Module — stakeholder data mastered and deduplicated via MDM

Configuration

Config Key	Description / Default
crm.health_score.threshold_warn	Score below which health is flagged — default 60
crm.engagement.default_cadence_days	Default stakeholder engagement cadence — default 30

Module 09: Organization Design System

Domain: Design | ume.domain.design | Module ID: ume.module.09

Purpose & Responsibility

Enterprise design system and design operations module. Maintains the organizational design token library, brand asset repository, UX pattern catalog, design review workflows, and provides design system governance across all digital products and communications.

- Maintain the authoritative design token library (colors, typography, spacing, motion)
- Govern the component and pattern library with versioning and deprecation management
- Manage brand asset repository: logos, icons, imagery, templates
- Coordinate design review and design approval workflows
- Track design system adoption and compliance across org products
- Support design sprint coordination and design research management

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Token Library	Design tokens	Color, typography, spacing, shadow, motion tokens with version control
Component Catalog	UI components	Documented, versioned components with usage guidelines and accessibility notes
Brand Asset Manager	Brand repository	Logo suite, color palette, imagery library, brand usage guidelines
Design Review Workflow	Approval process	Design review -> critique -> approve -> publish component lifecycle
Adoption Tracker	Compliance	Track which products and surfaces are using approved design system tokens
Research Repository	Design research	Store user research, usability test results, and insight records

Core Data Models

Model / Entity	Purpose & Key Fields
DesignToken { id, name, category, value, aliases, version, deprecated }	Design system token
Component { id, name, status, figma_ref, storybook_ref, version, a11y_score, tokens_used }	UI component record
BrandAsset { id, name, type, usage_rights, format, download_url, approved_uses }	Brand asset catalog entry

DesignReview { id, asset_id, reviewer_ids, status, comments, approved_at }	Design review instance
DesignResearch { id, title, method, participants, findings, recommendations, date }	Research record

Key Operations & API Surface

```
upsert_token(token: DesignToken) -> UmeResult<DesignToken>
publish_component(component: Component) -> UmeResult<Component>
upload_brand_asset(asset: BrandAsset) -> UmeResult<BrandAsset>
create_design_review(review: DesignReview) -> UmeResult<DesignReview>
get_token_snapshot() -> DesignTokenSnapshot
adoption_report(product_ids: Vec<String>) -> AdoptionReport
```

Domain Events

Event Topic	Trigger Condition
design.token.updated	Design token value or version changed
design.component.deprecated	UI component marked deprecated with migration guidance
design.review.approved	Design review approved for publication
design.brand_asset.published	New brand asset added to approved library

Integration Points

- CMS Module — brand assets stored as CMS content type
- Portal Module — design tokens drive portal UI theming
- PR & Branding Module — brand asset library shared with branding module
- Product Module — design system adoption tracked per product surface
- Templating System — design tokens and assets available in template engine

Configuration

Config Key	Description / Default
design.token_format	Export format for token consumers: css | json | scss — default json
design.review.required_approvers	Minimum approvers for component publication — default 2

Module 10: Engineering & Technology

Domain: Engineering | ume.domain.engineering | Module ID: ume.module.10

Purpose & Responsibility

Enterprise engineering and technology function management module. Coordinates engineering teams, manages the technology stack registry, tracks engineering initiatives, governs technical debt, manages engineering standards, and provides engineering productivity and health metrics.

- Maintain the technology stack registry with lifecycle and ownership metadata
- Track engineering initiatives, projects, and technical OKRs
- Manage engineering standards, architecture decision records (ADRs), and RFCs
- Monitor and govern technical debt inventory with prioritization scoring
- Coordinate code review standards, release management, and CI/CD pipeline health
- Produce engineering health metrics: velocity, cycle time, defect rates, coverage

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Tech Stack Registry	Technology catalog	Languages, frameworks, platforms, services with ownership and EOL tracking
Engineering Initiatives	Feature & project tracking	Engineering-led projects with OKRs, milestones, and delivery metrics
ADR / RFC Manager	Technical decisions	Architecture Decision Records and RFC lifecycle management
Tech Debt Tracker	Debt inventory	Catalogued technical debt items with effort estimate and priority score
Eng Standards Library	Standards & practices	Coding standards, security requirements, review checklists
SDLC Metrics	Delivery health	Velocity, lead time, MTTR, deployment frequency, change failure rate

Core Data Models

Model / Entity	Purpose & Key Fields
TechStackItem { id, name, type, version, owner_team, lifecycle_status, eol_date, risk }	Technology stack entry
EngineeringInitiative { id, title, type, owner_team, status, okr_ids, due_date }	Engineering initiative
ADR { id, title, status, context, decision, consequences, supersedes, created_at }	Architecture Decision Record

TechDebt { id, location, type, severity, effort_days, priority_score, created_at, resolved_at }	Technical debt item
SDLCMetric { team_id, period, velocity, lead_time_hours, mttr_hours, deploy_freq, cfr }	Engineering health metrics

Key Operations & API Surface

```
register_tech_stack_item(item: TechStackItem) -> UmeResult<TechStackItem>
create_adr(adr: ADR) -> UmeResult<ADR>
log_tech_debt(debt: TechDebt) -> UmeResult<TechDebt>
resolve_tech_debt(debt_id: &str, resolution: &str) -> UmeResult<()>
record_sdlc_metrics(metrics: SDLCMetric) -> UmeResult<()>
engineering_health_report(team_ids: Vec<String>) -> EngineeringHealthReport
```

Domain Events

Event Topic	Trigger Condition
eng.tech_stack.eol_approaching	Technology stack item nearing end-of-life date
eng.adr.approved	Architecture Decision Record formally approved
eng.tech_debt.critical_threshold	Tech debt score exceeded critical threshold
eng.sdlc.regression_detected	SDLC health metric regressed below baseline

Integration Points

- IT & Asset Management Module — tech stack items linked to IT asset records
- Product Module — engineering initiatives linked to product roadmap items
- Risk Management Module — tech stack EOL and high tech debt escalated as risks
- Requirements Module — engineering requirements linked to ADRs and initiatives
- Analytics Module — SDLC metrics feed engineering analytics dashboards

Configuration

Config Key	Description / Default
eng.tech_debt.critical_score	Tech debt priority score for critical escalation — default 80
eng.sdlc.velocity_baseline_window_weeks	Weeks of history for velocity baseline — default 12

Module 11: ESG, CSR & Sustainability

Domain: Sustainability | ume.domain.esg | Module ID: ume.module.11

Purpose & Responsibility

Enterprise ESG (Environmental, Social, Governance), CSR (Corporate Social Responsibility), and sustainability management module. Tracks environmental footprint, social impact programs, governance metrics, sustainability targets, and produces ESG reporting for disclosure frameworks including GRI, SASB, TCFD, and UN SDGs.

- Track environmental metrics: carbon emissions (Scope 1/2/3), energy, water, waste, biodiversity
- Manage CSR programs, community initiatives, and social impact measurement
- Monitor governance metrics: board diversity, executive pay ratio, ethics incidents
- Set and track sustainability targets with progress indicators
- Produce ESG disclosure reports aligned to GRI, SASB, TCFD, ISSB, and SDG frameworks
- Manage supplier ESG assessments and sustainability supply chain requirements

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Environmental Tracker	Carbon & resource data	Scope 1/2/3 emissions, energy, water, waste data entry and calculation
CSR Program Manager	Social initiatives	Community programs, volunteering, donations, impact measurement
Governance Metrics	ESG governance pillar	Board composition, pay equity, ethics incidents, anti-corruption KPIs
Sustainability Target Manager	Goal tracking	ESG commitments with milestones, owners, and progress indicators
Disclosure Engine	Report generation	Map org data to GRI, SASB, TCFD, ISSB, SDG disclosure templates
Supplier ESG Assessor	Supply chain ESG	ESG questionnaires, scores, and watchlist for supplier portfolio
Carbon Accounting	Emissions ledger	Structured carbon accounting ledger with audit trail and methodology notes

Core Data Models

Model / Entity	Purpose & Key Fields
EmissionRecord { id, scope, source, activity_data, emission_factor, co2e_kg, period, verified }	Carbon emission entry
CSRProgram { id, name, category, target_sdgs, budget_cents, impact_metrics, status }	CSR program record

ESGTarget { id, pillar, name, baseline, target_value, target_year, owner_id, progress }	Sustainability target
ESGDisclosure { id, framework, period, data_map: HashMap<String,JsonValue>, published_at }	ESG disclosure report
SupplierESGScore { supplier_id, score, tier, assessment_date, risks, improvements }	Supplier ESG assessment

Key Operations & API Surface

```
record_emission(e: EmissionRecord) -> UmeResult<EmissionRecord>
create_csr_program(p: CSRProgram) -> UmeResult<CSRProgram>
set_esg_target(t: ESGTarget) -> UmeResult<ESGTarget>
generate_disclosure(framework: ESGFramework, period: &str) -> UmeResult<ESGDisclosure>
assess_supplier_esg(supplier_id: &str, data: JsonValue) -> SupplierESGScore
carbon_footprint_summary(scope: EmissionScope, period: &str) -> CarbonSummary
```

Domain Events

Event Topic	Trigger Condition
esg.emission.recorded	Carbon emission data point added to ledger
esg.target.milestone_reached	ESG sustainability target milestone achieved
esg.target.at_risk	ESG target trajectory indicates miss risk
esg.disclosure.published	ESG disclosure report published
esg.supplier.failed_assessment	Supplier scored below acceptable ESG threshold

Integration Points

- Supply Chain Module — supplier data and procurement emissions data feeds
- Finance Module — ESG program costs linked to budget and reporting
- GRC Module — governance pillar metrics shared with governance framework
- Analytics Module — ESG KPIs surfaced in enterprise dashboard
- Operations Module — operational data feeds Scope 2 and 3 emissions calculation

Configuration

Config Key	Description / Default
esg.carbon.default_emission_factor_source	GHG Protocol source for emission factors — default ghg_protocol
esg.disclosure.default_framework	Primary ESG disclosure framework — default GRI
esg.supplier.minimum_score	Minimum acceptable supplier ESG score — default 60

Module 12: Enterprise Engineering, Management & Administration

Domain: Enterprise Management | ume.domain.enterprise_mgmt | Module ID: ume.module.12

Purpose & Responsibility

Meta-management module providing the enterprise-wide management plane for the UME OS itself. Coordinates inter-module governance, system-wide health and readiness reporting, enterprise program management, module configuration management, and provides the administrative interface for UME system operators.

- Provide the enterprise system overview: all module health, states, and metrics in one view
- Manage UME system-wide configuration lifecycle: staging, promotion, and rollback
- Coordinate enterprise-level programs spanning multiple organizational modules
- Govern module inter-dependencies and upgrade coordination
- Manage operator roles, access, and administrative actions
- Surface UME system diagnostics, bottlenecks, and health advisories to operators

Sub-components & Feature Areas

Sub-component	Feature Area	Description
System Overview	UME health dashboard	Aggregate health, state, and metrics from all registered modules
Config Lifecycle Mgr	Config management	Version-controlled config change management with approval gates
Enterprise Program Mgr	Cross-module programs	Programs spanning multiple modules with unified status tracking
Module Dependency Map	Dependency topology	Visualize and validate inter-module dependencies and upgrade order
Operator Console	Admin interface	Privileged operator commands: start/stop/restart modules, drain queues
System Diagnostics	Health advisor	Kernel health advisor: flag resource exhaustion, lock contention, debt

Core Data Models

Model / Entity	Purpose & Key Fields
SystemOverview { captured_at, modules: Vec<ModuleStatus>, kernel_health, alerts }	Enterprise system snapshot
ConfigChange { id, target, before: JsonValue, after: JsonValue, approved_by, applied_at }	Config change record

EnterpriseProgram { id, name, modules_involved, status, milestones, exec_sponsor }	Cross-module enterprise program
SystemDiagnostic { id, severity, category, message, suggested_action, detected_at }	System health advisory

Key Operations & API Surface

```
get_system_overview() -> SystemOverview
propose_config_change(change: ConfigChange) -> UmeResult<ConfigChange>
apply_config_change(change_id: &str) -> UmeResult<()>
rollback_config_change(change_id: &str) -> UmeResult<()>
create_enterprise_program(p: EnterpriseProgram) -> UmeResult<EnterpriseProgram>
run_diagnostics() -> Vec<SystemDiagnostic>
restart_module(module_id: &str) -> UmeResult<()>
```

Domain Events

Event Topic	Trigger Condition
enterprise.config.applied	System configuration change applied
enterprise.config.rolled_back	Configuration change rolled back
enterprise.diagnostic.critical	Critical system diagnostic advisory raised
enterprise.module.restarted	Module restart initiated by operator

Integration Points

- All org modules — system overview pulls health reports from each module
- Backup Module — config state included in system snapshots
- Audit Module — all operator actions produce mandatory audit records
- Security Module — operator access governed by strict RBAC policies

Configuration

Config Key	Description / Default
enterprise_mgmt.overview_refresh_sec	System overview refresh interval — default 30
enterprise_mgmt.config_change.require_approval	Require approval before config changes applied — default true

Module 13: Legal Entity Management (Chombo)

Domain: Legal | ume.domain.chombo | Module ID: ume.module.13

Purpose & Responsibility

Authoritative legal entity management subsystem. Manages the full lifecycle of all legal entities within the organizational group — registrations, corporate structures, jurisdiction compliance, filing plans, governance profiles, and inter-entity relationships. Policy pack-driven compliance evaluation ensures continuous regulatory alignment.

- Register and maintain all legal entities: companies, trusts, partnerships, branches
- Evaluate entity compliance against 45+ jurisdiction-aware policy packs
- Generate and track statutory filing plans and filing outcomes
- Manage inter-entity relationship graph: parent, subsidiary, associate, JV structures
- Maintain governance profiles: board composition, signatories, authorized representatives
- Coordinate deregistration, dormancy, and dissolution workflows

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Entity Registry	Master entity store	LegalEntityRecord store via LegalEntityRepository trait
Policy Pack Engine	Compliance evaluation	45 policy packs (pack_001..pack_045) with LegalPolicyPack trait
Filing Planner	Statutory schedule	Build and track filing schedules via FilingRepository
Relationship Manager	Entity graph	Inter-entity ownership and structural relationships
Governance Profile Manager	Board & officers	Director records, signatory registers, authorized officers
Jurisdiction Policy Library	Regulatory rules	JurisdictionPolicy definitions per region and entity type
Entity Workflow Engine	Lifecycle workflows	collect-docs -> run-kyc -> register workflow orchestration

Core Data Models

Model / Entity	Purpose & Key Fields
LegalEntityRecord { id, name, type, registration_number, jurisdiction, status, metadata }	Primary legal entity aggregate
ComplianceAlert { entity_id, pack_id, rule_id, severity, message, remediation }	Policy pack compliance alert
FilingRecord { id, entity_id, pack_id, filing_type, due_date, status, filed_at }	Statutory filing record

RelationshipRecord { id, parent_id, child_id, type, ownership_pct, effective_date }	Inter-entity relationship
GovernanceProfile { entity_id, directors, signatories, authorized_officers, last_updated }	Entity governance record
JurisdictionPolicy { id, jurisdiction, entity_types, rules: Vec<PolicyRule> }	Jurisdiction-specific policy
ChomboEvaluation { entity_id, alerts: Vec<ComplianceAlert>, filings: Vec<FilingRecord> }	Evaluation result envelope

Key Operations & API Surface

```
register_entity(entity: LegalEntityRecord) -> UmeResult<LegalEntityRecord>
evaluate_entity(entity_id: &str) -> ChomboEvaluation
schedule_filings(entity_id: &str) -> Vec<FilingRecord>
add_relationship(rel: RelationshipRecord) -> UmeResult<()>
update_governance_profile(profile: GovernanceProfile) -> UmeResult<()>
policy_pack_count() -> usize
entity_compliance_summary() -> ComplianceSummary
```

Domain Events

Event Topic	Trigger Condition
chombo.entity.registered	Legal entity added to registry
chombo.entity.evaluated	Compliance evaluation completed — includes alert count
chombo.filing.due	Filing deadline approaching (configurable lead days)
chombo.filing.overdue	Filing deadline passed without recorded completion
chombo.compliance.alert.critical	Critical compliance alert generated by policy pack

Integration Points

- Board Management Module — board composition data linked to governance profiles
- Finance Module — entity tax profiles and regulatory fees linked to accounts
- GRC Module — compliance alerts feed GRC risk and control registers
- Administration Module — entity structure feeds org unit hierarchy
- MDM Module — legal entity master data governed through MDM

Configuration

Config Key	Description / Default
chombo.filing.lead_days	Days before filing due to emit warning event — default 30

chombo.evaluation.auto_on_register	Auto-evaluate entity on registration — default true

Module 14: Finance & Accounting

Domain: Finance | ume.domain.finance | Module ID: ume.module.14

Purpose & Responsibility

Enterprise finance and accounting module providing the general ledger, accounts payable/receivable, budgeting, financial planning and analysis (FP&A), tax management, treasury operations, and financial reporting capabilities. Serves as the financial backbone of the organization OS.

- Maintain the multi-entity general ledger with full double-entry accounting
- Manage accounts payable, accounts receivable, and cash application
- Produce budgets, forecasts, and variance analysis
- Handle tax calculations, filings coordination, and transfer pricing records
- Manage treasury: cash pooling, FX exposure, banking relationships
- Generate statutory and management financial statements with consolidation

Sub-components & Feature Areas

Sub-component	Feature Area	Description
General Ledger	Chart of accounts & journals	Double-entry journal posting; multi-entity, multi-currency COA
AP / AR Engine	Payables & receivables	Invoice lifecycle, payment terms, aging analysis, dunning
Budget Manager	Budget planning	Annual budget templates, cost center allocations, budget vs actuals
FP&A Engine	Forecasting	Rolling forecasts, scenario planning, revenue/cost drivers
Tax Management	Tax compliance	VAT/GST, corporate tax, withholding tax, tax register
Treasury Manager	Cash & FX	Cash pooling, liquidity forecasting, FX exposure, bank reconciliation
Financial Reporting	Statutory & management reports	P&L, balance sheet, cash flow statement, consolidation

Core Data Models

Model / Entity	Purpose & Key Fields
JournalEntry { id, entity_id, period, lines: Vec<JournalLine>, status, reference, posted_at }	GL journal entry
Invoice { id, type, entity_id, counterparty_id, amount_cents, currency, status, due_date }	AP/AR invoice

BudgetLine { id, entity_id, cost_center, account, period, budget_cents, forecast_cents }	Budget allocation line
TaxRecord { id, entity_id, tax_type, jurisdiction, period, amount_cents, status, filed_at }	Tax register entry
TreasuryCashFlow { id, entity_id, type, amount_cents, currency, value_date, bank_account }	Treasury cash flow
FinancialStatement { id, entity_id, type, period, data: JsonValue, consolidated, generated_at }	Financial statement

Key Operations & API Surface

```
post_journal_entry(entry: JournalEntry) -> UmeResult<JournalEntry>
create_invoice(invoice: Invoice) -> UmeResult<Invoice>
settle_invoice(invoice_id: &str, payment: PaymentRecord) -> UmeResult<()>
set_budget(lines: Vec<BudgetLine>) -> UmeResult<()>
get_budget_vs_actuals(entity_id: &str, period: &str) -> BudgetVarianceReport
record_tax(record: TaxRecord) -> UmeResult<TaxRecord>
generate_financial_statement(entity_id: &str, type: StatementType, period: &str) ->
FinancialStatement
```

Domain Events

Event Topic	Trigger Condition
finance.journal.posted	Journal entry posted to general ledger
finance.invoice.overdue	Invoice payment overdue — dunning triggered
finance.budget.variance_exceeded	Actual cost exceeded budget by configured threshold
finance.statement.generated	Financial statement produced for period
finance.tax.filing_due	Tax filing deadline approaching

Integration Points

- Legal Entity Module — multi-entity accounting aligned to legal entity structure

Configuration

Config Key	Description / Default
finance.currency.base	Base reporting currency — default USD
finance.budget.variance_threshold_pct	Variance % triggering alert — default 10

finance.ap.default_payment_terms_days	Default AP payment terms — default 30

Module 15: Governance, Risk & Compliance (GRC)

Domain: GRC | ume.domain.grc | Module ID: ume.module.15

Purpose & Responsibility

Enterprise GRC module providing a unified framework for organizational governance, risk management, and regulatory compliance. Maintains the risk register, control library, compliance obligation tracker, policy compliance assurance, internal audit management, and regulatory change monitoring.

- Maintain the enterprise risk register with risk owners, treatments, and residual risk scoring
- Manage the control library with control design assessments and operating effectiveness testing
- Track all regulatory and contractual compliance obligations with evidence management
- Coordinate internal audit lifecycle: planning, fieldwork, findings, recommendations, remediation
- Monitor regulatory change and perform obligation impact analysis
- Produce board-level GRC dashboards and regulatory reporting

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Risk Register	Risk inventory	Risk identification, scoring (likelihood x impact), treatment, residual risk
Control Library	Control management	Control descriptions, design assessments, testing schedules, deficiencies
Compliance Obligation Tracker	Regulatory obligations	Laws, regulations, standards, contractual commitments with requirement mapping
Internal Audit Manager	Audit lifecycle	Audit plan -> fieldwork -> findings -> remediation -> closure
Policy Compliance Assurance	Policy monitoring	Test compliance with internal administrative policies
Regulatory Change Monitor	Change intelligence	Track regulatory changes; assess impact on obligation inventory
GRC Reporting	Board & regulator reports	Risk heatmaps, control effectiveness, compliance status dashboards

Core Data Models

Model / Entity	Purpose & Key Fields
Risk { id, title, category, likelihood, impact, score, owner_id, treatment, residual_score, review_date }	Risk register entry
Control { id, risk_ids, description, type, frequency, owner_id, design_rating, operating_rating }	Control record

Obligation { id, source, requirement, domain, owner_id, evidence_ids, status, next_review }	Compliance obligation
AuditEngagement { id, scope, type, auditor_ids, status, findings: Vec<AuditFinding>, report_date }	Internal audit engagement
AuditFinding { id, engagement_id, title, severity, root_cause, recommendation, status, due_date }	Audit finding record
RegulatoryChange { id, source, description, effective_date, impact_assessment, obligation_ids }	Regulatory change entry

Key Operations & API Surface

```
register_risk(risk: Risk) -> UmeResult<Risk>
evaluate_risk(risk_id: &str) -> RiskEvaluation
register_control(control: Control) -> UmeResult<Control>
record_control_test(control_id: &str, result: ControlTestResult) -> UmeResult<()>
add_obligation(obligation: Obligation) -> UmeResult<Obligation>
create_audit_engagement(engagement: AuditEngagement) -> UmeResult<AuditEngagement>
grc_dashboard() -> GRCDashboard
```

Domain Events

Event Topic	Trigger Condition
grc.risk.critical_threshold	Risk score exceeded critical threshold
grc.control.deficiency_identified	Control test resulted in deficiency finding
grc.obligation.overdue	Compliance obligation evidence overdue
grc.audit.finding.raised	New audit finding raised in active engagement
grc.regulatory_change.impact_assessed	Regulatory change impact assessment completed

Integration Points

- Legal Entity Module — compliance alerts from Chombo feed GRC obligation register
- Finance Module — financial controls and SOX-type controls managed in GRC
- Security Module — security risks and controls linked to GRC risk register
- Board Management Module — GRC dashboard surfaced in board pack
- Administration Module — internal policies tested for compliance via GRC

Configuration

Config Key	Description / Default

grc.risk.critical_score	Risk score defining critical threshold — default 15 (5x5 matrix)
grc.control.test_frequency_days	Default control test frequency — default 90
grc.obligation.review_lead_days	Days before obligation review to trigger reminder — default 14

Module 16: Human Resources (HR & People)

Domain: People | ume.domain.hr | Module ID: ume.module.16

Purpose & Responsibility

Comprehensive human resources and people management module. Manages the full employee lifecycle from recruitment through separation, payroll computation, benefits administration, performance management, workforce planning, and people analytics. Serves as the authoritative people data source for the organization OS.

- Maintain the master employee registry with full employment lifecycle records
- Manage recruitment: job requisitions, candidates, offers, onboarding
- Handle payroll computation, benefits enrollment, and tax profiles
- Coordinate performance reviews, development plans, and succession planning
- Manage leave, timekeeping, and workforce scheduling
- Produce workforce analytics: headcount, turnover, diversity, cost-per-head

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Employee Registry	People master data	Authoritative employee records; employment history; org unit links
Recruitment Engine	Talent acquisition	Job requisitions, applicant tracking, interview scheduling, offers
Payroll Engine	Pay computation	Gross-to-net payroll, statutory deductions, pay slip generation
Benefits Manager	Benefits administration	Enrollment, eligibility, provider management, benefit statements
Performance Manager	Reviews & development	Review cycles, 360 feedback, development plans, succession maps
Time & Leave Manager	Attendance & absence	Leave requests, approvals, timekeeping entries, scheduling
Workforce Analytics	People metrics	Headcount, turnover, diversity KPIs, cost-per-head, engagement scores

Core Data Models

Model / Entity	Purpose & Key Fields
Employee { id, full_name, email, role, manager_id, org_unit_id, employment_status, hire_date }	Employee master record
Requisition { id, title, org_unit_id, headcount, status, owner_id, target_date }	Job requisition

Candidate { id, requisition_id, name, stage, score, offer_id, status }	Recruitment candidate
PayrollEntry { id, employee_id, period, gross_cents, deductions_cents, net_cents, paid_at }	Payroll computation record
BenefitEnrollment { id, employee_id, benefit_type, provider, start_date, end_date, cost_cents }	Benefit enrollment
PerformanceReview { id, employee_id, reviewer_id, period, rating, comments, completed_at }	Performance review record
LeaveRequest { id, employee_id, type, start_date, end_date, status, approved_by }	Leave request

Key Operations & API Surface

```
upsert_employee(e: Employee) -> UmeResult<Employee>
create_requisition(r: Requisition) -> UmeResult<Requisition>
process_payroll(org_unit_id: &str, period: &str) -> Vec<PayrollEntry>
upsert_benefit(enrollment: BenefitEnrollment) -> UmeResult<BenefitEnrollment>
upsert_review(review: PerformanceReview) -> UmeResult<PerformanceReview>
submit_leave_request(req: LeaveRequest) -> UmeResult<LeaveRequest>
workforce_summary() -> WorkforceSummary
```

Domain Events

Event Topic	Trigger Condition
hr.employee.hired	New employee record created and activated
hr.employee.separated	Employee separation recorded
hr.payroll.run_completed	Payroll cycle computation completed
hr.review.completed	Performance review cycle closed
hr.leave.approved	Leave request approved

Integration Points

- Administration Module — employee org unit links feed org hierarchy
- Finance Module — payroll outputs feed AP and general ledger
- IT & Asset Management — employee onboarding triggers IT provisioning workflows
- Learning Management — training requirements linked to performance development plans
- Analytics Module — workforce metrics fed into enterprise analytics

Configuration

Config Key	Description / Default
hr.payroll.frequency	Pay frequency: weekly | bi_weekly | monthly — default monthly
hr.review.cycle_months	Performance review cycle frequency in months — default 12
hr.leave.carryover_days	Maximum leave days carryover per year — default 5

Module 17: Investment Management

Domain: Investment | ume.domain.investment | Module ID: ume.module.17

Purpose & Responsibility

Enterprise investment and capital allocation management module. Manages the organizational investment portfolio including capital projects, financial securities, real estate, equity stakes, and alternative investments. Provides investment appraisal, portfolio performance tracking, and capital allocation governance.

- Maintain the enterprise investment portfolio across all asset classes
- Conduct investment appraisal: DCF, IRR, NPV, payback period analysis
- Track portfolio performance against benchmarks and strategic targets
- Manage capital allocation committees and investment approval workflows
- Coordinate due diligence processes for new investment opportunities
- Produce investment performance reports for board and external stakeholders

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Portfolio Registry	Investment catalog	All investment positions with asset class, cost, current value, returns
Appraisal Engine	Financial modeling	DCF, IRR, NPV, payback, sensitivity analysis for investment evaluation
Performance Tracker	Returns & benchmarks	Time-weighted returns, benchmarks, attribution, risk-adjusted metrics
Capital Allocation Committee	Governance workflow	Investment approval workflow with committee voting and authorization
Due Diligence Manager	DD coordination	Structured due diligence templates, findings, and sign-off tracking
Investment Reporting	Performance reports	Portfolio dashboards, board investment reports, regulatory disclosures

Core Data Models

Model / Entity	Purpose & Key Fields
Investment { id, name, type, cost_cents, current_value_cents, irr_pct, status, acquired_at }	Investment position
Appraisal { id, investment_id, method, assumptions: JsonValue, result: JsonValue, analyst_id }	Investment appraisal
AllocationDecision { id, investment_id, amount_cents,	Capital allocation approval

approved_by, approved_at, conditions }	
DDFinding { id, investment_id, category, risk_level, finding, recommendation, resolved }	Due diligence finding
PortfolioPerformance { period, total_value_cents, twr_pct, benchmark_pct, alpha_pct }	Portfolio performance snapshot

Key Operations & API Surface

```
register_investment(inv: Investment) -> UmeResult<Investment>
run_appraisal(inv_id: &str, method: AppraisalMethod, assumptions: JsonValue) -> Appraisal
submit_for_allocation(inv_id: &str, amount_cents: i64) -> UmeResult<AllocationDecision>
record_dd_finding(finding: DDFinding) -> UmeResult<DDFinding>
portfolio_performance(period: &str) -> PortfolioPerformance
investment_report(filter: InvestmentFilter) -> InvestmentReport
```

Domain Events

Event Topic	Trigger Condition
investment.approved	Capital allocation decision approved by committee
investment.performance.below_benchmark	Investment return dropped below benchmark threshold
investment.dd.critical_finding	Due diligence critical finding logged
investment.divested	Investment position exited and removed from portfolio

Integration Points

- Finance Module — investment valuations and returns feed financial statements
- Risk Management Module — investment risk exposures feed risk register
- Board Management Module — investment decisions require board or committee approval
- Business Development Module — BD opportunities evaluated via investment appraisal engine

Configuration

Config Key	Description / Default
investment.discount_rate_pct	Default WACC for DCF appraisals — default 10
investment.approval_threshold_cents	Amount above which committee approval is required — default 50000000

Module 18: IT & Asset Management

Domain: IT | ume.domain.it | Module ID: ume.module.18

Purpose & Responsibility

Comprehensive IT operations and enterprise asset management module. Manages the full lifecycle of IT assets (hardware, software, cloud, licenses), IT service management (ITSM), infrastructure configuration, and IT financial management. Provides the authoritative CMDB (Configuration Management Database) for the organization.

- Maintain the CMDB: all configuration items (CIs) with relationships and dependency maps
- Manage IT asset lifecycle: procurement -> deployment -> maintenance -> retirement
- Handle ITSM: incident management, problem management, change management, service requests
- Track software license compliance and software asset management (SAM)
- Manage cloud resource inventory, cost allocation, and FinOps
- Produce IT capacity planning reports and technology refresh plans

Sub-components & Feature Areas

Sub-component	Feature Area	Description
CMDB	Configuration management	CIs, relationships, dependency maps, impact analysis
Asset Lifecycle Manager	Asset management	Procurement to retirement asset tracking with financial values
ITSM Engine	Service management	Incidents, problems, changes, service requests with SLA tracking
SAM / License Manager	Software licensing	License inventory, compliance checks, renewal tracking
Cloud Resource Manager	Cloud FinOps	Cloud resource inventory, tagging, cost allocation, waste detection
IT Capacity Planner	Capacity & refresh	Capacity utilization, growth projections, refresh planning
IT Service Catalog	Service portfolio	Catalog of available IT services with SLAs and request workflows

Core Data Models

Model / Entity	Purpose & Key Fields
ConfigurationItem { id, name, type, status, owner_id, location, dependencies: Vec<String> }	CMDB CI record
ITAsset { id, ci_id, purchase_date, cost_cents,	IT asset financial record

depreciation_method, retirement_date, warranty }	
Incident { id, title, severity, status, assignee, ci_ids, opened_at, resolved_at, sla_breached }	IT incident record
SoftwareLicense { id, product, vendor, count, used, expiry_date, cost_cents, compliance_status }	Software license record
CloudResource { id, provider, type, region, account, monthly_cost_cents, tags, waste_flag }	Cloud resource entry
ChangeRequest { id, title, type, risk, ci_ids, schedule, approver_ids, status, cab_approved }	IT change request

Key Operations & API Surface

```
register_ci(ci: ConfigurationItem) -> UmeResult<ConfigurationItem>
create_incident(incident: Incident) -> UmeResult<Incident>
resolve_incident(incident_id: &str, resolution: &str) -> UmeResult<()>
create_change_request(cr: ChangeRequest) -> UmeResult<ChangeRequest>
check_license_compliance() -> LicenseComplianceReport
cloud_cost_summary(account: &str, period: &str) -> CloudCostSummary
it_service_catalog() -> Vec<ITService>
```

Domain Events

Event Topic	Trigger Condition
it.incident.critical_sla_breach	Critical incident SLA breach detected
it.change.approved	Change request approved by CAB
it.license.compliance_violation	Software license compliance violation detected
it.asset.retirement_due	IT asset approaching scheduled retirement date
it.cloud.waste_detected	Cloud resource waste pattern identified

Integration Points

- Security Module — security vulnerabilities and patches managed as change requests
- Engineering Module — tech stack registry synced with CMDB CIs
- HR Module — employee onboarding triggers IT provisioning from service catalog
- Finance Module — IT asset depreciation feeds financial accounting
- GRC Module — IT risks and controls linked to GRC register

Configuration

Config Key	Description / Default
it.incident.sla.p1_hours	P1 incident SLA resolution target in hours — default 4
it.cloud.waste_threshold_pct	Monthly unused cloud resource % triggering waste flag — default 20
it.license.alert_days_before_expiry	Days before license expiry to alert — default 60

Module 19: Enterprise Knowledge Management System (KMS)

Domain: Knowledge | ume.domain.knowledge | Module ID: ume.module.19

Purpose & Responsibility

Enterprise knowledge management system providing a structured, searchable, and governed organizational knowledge base. Manages explicit knowledge (articles, how-tos, reference material, lessons learned) and supports the capture and codification of tacit knowledge through structured templates and expert elicitation workflows.

- Maintain the enterprise knowledge repository with classification and tagging
- Manage knowledge article lifecycle: draft -> expert review -> publish -> maintain -> retire
- Provide powerful search and recommendation across all knowledge assets
- Capture lessons learned from projects, incidents, and evaluations
- Manage subject matter expert directories and knowledge owner assignments
- Track knowledge gaps and trigger knowledge creation campaigns

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Knowledge Repository	Article storage	Structured knowledge articles with rich metadata via CMS integration
Classification Engine	Taxonomy & tagging	Hierarchical taxonomy with auto-classification suggestions
Expert Directory	SME management	Subject matter expert profiles, areas of expertise, availability
Lessons Learned Tracker	Post-event capture	Structured lessons learned capture linked to projects, incidents, evaluations
Knowledge Search	Discovery	AI-enhanced semantic search across all knowledge assets
Gap Analyzer	Knowledge gap detection	Identify areas of insufficient knowledge coverage from query analysis
Knowledge Analytics	Consumption metrics	Article views, search hit rate, gap coverage progress, contribution metrics

Core Data Models

Model / Entity	Purpose & Key Fields
KnowledgeArticle { id, title, type, author_id, reviewer_id, status, tags, content_asset_id }	Knowledge article record
Taxonomy { id, name, parent_id, description, article_count }	Knowledge taxonomy node

Expert { employee_id, domains: Vec<String>, availability, articles_authored, rating }	SME profile
LessonLearned { id, source_type, source_id, lesson, recommendation, captured_by, tags }	Lessons learned entry
KnowledgeGap { id, domain, query_pattern, gap_score, suggested_action, created_at }	Detected knowledge gap

Key Operations & API Surface

```
create_article(article: KnowledgeArticle) -> UmeResult<KnowledgeArticle>
publish_article(article_id: &str) -> UmeResult<()>
search_knowledge(query: KnowledgeQuery) -> Vec<KnowledgeSearchResult>
capture_lesson_learned(lesson: LessonLearned) -> UmeResult<LessonLearned>
find_expert(domain: &str) -> Vec<Expert>
identify_knowledge_gaps() -> Vec<KnowledgeGap>
knowledge_analytics() -> KnowledgeAnalytics
```

Domain Events

Event Topic	Trigger Condition
knowledge.article.published	Knowledge article moved to published state
knowledge.article.outdated	Article flagged as potentially outdated based on age threshold
knowledge.gap.identified	New knowledge gap detected from query analysis
knowledge.lesson.captured	Lessons learned entry added to repository

Integration Points

- CMS Module — knowledge articles stored as CMS content assets
- HR Module — SME directory linked to employee profiles
- Analytics Module — knowledge consumption metrics aggregated in enterprise analytics
- Learning Management Module — knowledge articles surfaced as reference material in learning paths
- Process Module — process documentation linked to relevant knowledge articles

Configuration

Config Key	Description / Default
knowledge.article.review_cycle_days	Article review frequency — default 365
knowledge.gap.min_query_count	Minimum query count to flag a gap — default 10

knowledge.search.similarity_threshold	Semantic search similarity floor — default 0.6

Module 20: Learning & Development Management

Domain: Learning | ume.domain.learning | Module ID: ume.module.20

Purpose & Responsibility

Enterprise learning management system (LMS) module supporting the full learning lifecycle: learning content library, course design, learning path management, learner progress tracking, compliance training management, skills gap analysis, and learning effectiveness measurement.

- Maintain the enterprise learning content library with version management
- Design and manage learning paths and curricula by role, level, and domain
- Track learner enrollment, progress, completion, and assessment scores
- Manage mandatory compliance training with completion tracking and escalation
- Conduct skills gap analysis and produce personalized learning recommendations
- Measure training effectiveness: knowledge transfer, behavior change, business impact

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Content Library	Learning content management	Courses, modules, videos, SCORM/xAPI packages, assessments
Curriculum Designer	Learning path builder	Role-based learning paths, prerequisites, completion criteria
Enrollment Engine	Learner management	Enrollments, waitlists, cohorts, scheduled vs self-paced modes
Progress Tracker	Completion & assessment	Module completion, quiz scores, certificates, CPD points
Compliance Training Manager	Mandatory training	Assign, track, escalate, and report on compliance training obligations
Skills Inventory	Competency framework	Skill definitions, proficiency levels, employee skill assessments
Learning Analytics	Effectiveness metrics	Completion rates, assessment scores, NPS, Kirkpatrick ROI metrics

Core Data Models

Model / Entity	Purpose & Key Fields
LearningContent { id, title, type, format, duration_min, version, tags, content_asset_id }	Learning content item
LearningPath { id, name, target_role, items: Vec<PathItem>, total_hours, required }	Learning path definition

Enrollment { id, learner_id, content_id, status, enrolled_at, completed_at, score }	Learner enrollment record
ComplianceTraining { id, content_id, required_roles, deadline_rule, escalation_contacts }	Compliance training assignment
Skill { id, name, domain, proficiency_levels: Vec<String>, related_roles }	Skill definition
SkillAssessment { employee_id, skill_id, current_level, assessed_by, assessed_at, gap_to_required }	Employee skill assessment

Key Operations & API Surface

```
publish_content(content: LearningContent) -> UmeResult<LearningContent>
create_learning_path(path: LearningPath) -> UmeResult<LearningPath>
enroll_learner(enrollment: Enrollment) -> UmeResult<Enrollment>
record_progress(learner_id: &str, content_id: &str, progress: ProgressUpdate) ->
UmeResult<()>
assign_compliance_training(assignment: ComplianceTraining) -> UmeResult<()>
assess_skill_gaps(employee_id: &str) -> Vec<SkillGap>
learning_analytics(filter: LearningFilter) -> LearningAnalytics
```

Domain Events

Event Topic	Trigger Condition
learning.content.completed	Learner completed learning content item
learning.compliance.overdue	Mandatory compliance training not completed before deadline
learning.skill_gap.identified	Skill gap identified for employee or org unit
learning.certificate.issued	Completion certificate issued to learner

Integration Points

- HR Module — employee roles drive automatic learning path assignment

Configuration

Config Key	Description / Default
learning.compliance.escalation_days	Days past deadline before escalation — default 7
learning.content.expiry_months	Learning content review frequency — default 12
learning.skill.gap_alert_threshold	Proficiency gap level that triggers learning alert — default 2

Module 21: Management & Strategy

Domain: Strategy | ume.domain.strategy | Module ID: ume.module.21

Purpose & Responsibility

Enterprise strategy formulation, deployment, and performance management module. Manages the full strategy lifecycle from environmental scanning and strategic positioning through OKR/KPI setting, strategy execution monitoring, strategic initiative management, and board-level strategy reviews.

- Maintain the enterprise strategy architecture: vision, mission, strategic pillars, and objectives
- Manage the OKR/KPI framework with cascading from enterprise to team level
- Coordinate strategic initiative portfolio with owner accountability
- Conduct strategy execution reviews: monthly, quarterly, and annual cadences
- Integrate environmental scanning (PESTLE, SWOT, Porter) into strategy refresh
- Produce strategy scorecards and balanced scorecard reporting

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Strategy Architecture	Strategic framework	Vision, mission, strategic pillars, long-term objectives, horizon planning
OKR / KPI Manager	Goal management	Cascading OKRs, KPI definitions, targets, actuals, scoring
Strategic Initiative Tracker	Execution management	Strategic initiatives with owners, milestones, resources, status
Environmental Scanning	PESTLE & SWOT	PESTLE factors, SWOT analyses, competitive signals, scenario inputs
Strategy Review Engine	Performance reviews	Cadenced strategy reviews with structured check-in and update workflows
Balanced Scorecard	Performance reporting	BSC perspectives: Financial, Customer, Process, Learning & Growth
Strategy Analytics	Insights	OKR completion rates, initiative on-track ratios, strategic risk indicators

Core Data Models

Model / Entity	Purpose & Key Fields
StrategicObjective { id, pillar, title, owner_id, horizon, status, kpi_ids }	Strategic objective
OKR { id, objective_id, key_result, target, current, unit, period, confidence_pct }	OKR key result
StrategicInitiative { id, title, objective_id, owner_id, status, budget_cents, milestones }	Strategic initiative

PESTLEAnalysis { id, period, political, economic, social, tech, legal, environmental, implications }	PESTLE analysis
SWOTAnalysis { id, period, strengths, weaknesses, opportunities, threats, strategic_options }	SWOT analysis
StrategyReview { id, period, type, attendees, okr_updates, initiative_updates, decisions }	Strategy review session

Key Operations & API Surface

```
set_strategic_objective(obj: StrategicObjective) -> UmeResult<StrategicObjective>
create_okr(okr: OKR) -> UmeResult<OKR>
update_okr_progress(okr_id: &str, current: f64, notes: &str) -> UmeResult<()>
create_strategic_initiative(init: StrategicInitiative) -> UmeResult<StrategicInitiative>
run_pestle_analysis(analysis: PESTLEAnalysis) -> UmeResult<PESTLEAnalysis>
strategy_scorecard(period: &str) -> StrategyScorecard
strategy_review_report(period: &str) -> StrategyReviewReport
```

Domain Events

Event Topic	Trigger Condition
strategy.okr.at_risk	OKR confidence score dropped below threshold
strategy.initiative.milestone_missed	Strategic initiative milestone not completed on time
strategy.review.completed	Strategy review session completed and decisions recorded
strategy.objective.achieved	Strategic objective fully achieved

Integration Points

- Analytics Module — OKR metrics fed from enterprise analytics KPI engine
- Board Management Module — strategy reviews surfaced in board packs
- Business Development Module — BD pipeline aligned to strategic objectives
- Finance Module — strategic initiative budgets funded through finance module
- All org modules — OKRs cascade to domain-specific module KPIs

Configuration

Config Key	Description / Default
strategy.okr.at_risk_confidence_pct	OKR confidence below which at-risk event is emitted — default 40
strategy.review.quarterly_reminder_days	Days before quarterly review to trigger reminder — default 14
strategy.okr.scoring.period_months	OKR scoring period in months — default 3

Module 22: Marketing System (Soko)

Domain: Marketing | ume.domain.soko | Module ID: ume.module.22

Purpose & Responsibility

Enterprise marketing system providing campaign orchestration, market signal processing, strategy evaluation, and activation planning. Built on the Soko subsystem with 70 strategy packs. Manages the full marketing lifecycle from audience discovery through campaign launch, optimization, and performance reporting.

- Register and manage marketing campaigns with full metadata, channels, and budget allocation
- Ingest real-time market signals and behavioral telemetry
- Evaluate campaigns through 70 strategy packs for insights and activation actions
- Manage audience personas, channel allocations, and content asset libraries
- Coordinate campaign workflow: audience-discovery -> creative-assembly -> launch -> optimization
- Produce marketing performance reports: ROI, CAC, CLV, channel effectiveness

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Campaign Manager	Campaign lifecycle	Full campaign metadata, budget, channels, status, and timeline management
Signal Processor	Market telemetry	Real-time ingestion of behavioral, transactional, and market signals
Strategy Pack Engine	70 strategy packs	Parallel evaluation of 70 strategy packs for insights and actions
Audience Manager	Persona & segmentation	Audience persona library, segmentation rules, audience size estimation
Content Asset Library	Marketing content	Campaign creative assets linked to CMS and design system
Campaign Workflow	Execution coordination	Stage-gated campaign workflow with execution notes generation
Marketing Analytics	Performance reporting	ROI, CAC, CLV, channel attribution, conversion funnel analytics

Core Data Models

Model / Entity	Purpose & Key Fields
MarketingCampaign { id, name, type, channels, budget_cents, start_date, end_date, status }	Campaign master record
MarketSignal { id, campaign_id, signal_type, source, value, captured_at }	Market telemetry signal

StrategyInsight { campaign_id, pack_id, code, severity, message, recommendation }	Strategy pack insight
ActivationAction { campaign_id, pack_id, action_type, description, priority, estimated_lift_pct }	Recommended activation action
CampaignEvaluation { campaign_id, insights: Vec<StrategyInsight>, actions: Vec<ActivationAction> }	Evaluation result
AudiencePersona { id, name, demographics, behavioral_traits, channel_preferences, size_estimate }	Audience persona
ChannelAllocation { campaign_id, channel, budget_pct, expected_reach, expected_conversions }	Channel budget allocation

Key Operations & API Surface

```
register_campaign(campaign: MarketingCampaign) -> UmeResult<MarketingCampaign>
ingest_signal(signal: MarketSignal) -> UmeResult<()>
evaluate_campaign(campaign_id: &str) -> CampaignEvaluation
strategy_pack_count() -> usize
soko_summary() -> SokoSummary
campaign_performance_report(campaign_id: &str) -> CampaignPerformanceReport
list_campaigns(filter: CampaignFilter) -> Vec<MarketingCampaign>
```

Domain Events

Event Topic	Trigger Condition
soko.campaign.registered	New marketing campaign registered
soko.campaign.evaluated	Campaign strategy evaluation completed with insights
soko.signal.ingested	Market signal ingested to campaign
soko.campaign.budget_exceeded	Campaign spend exceeded configured threshold
soko.campaign.completed	Campaign lifecycle reached completion

Integration Points

- CRM Module — campaign audiences linked to CRM stakeholder segments
- Sales Module — marketing campaign leads flow to sales pipeline
- Analytics Module — campaign performance data feeds enterprise analytics
- Communications Module — campaign messaging dispatched via communications module
- Design Module — campaign creative assets from design system brand library

Configuration

Config Key	Description / Default
soko.strategy_packs.enabled_count	Number of strategy packs to run per evaluation — default all 70
soko.signal.retention_days	Signal retention window — default 90
soko.campaign.budget_alert_threshold_pct	% of budget spent before alert — default 80

Module 23: Master Data Management (MDM)

Domain: Data | ume.domain.mdm | Module ID: ume.module.23

Purpose & Responsibility

Enterprise master data management module providing authoritative golden records across all critical business entities: organizations, individuals, products, locations, accounts, and assets. Delivers deduplication, data quality management, hierarchical data governance, and master data distribution to all consuming modules.

- Define and govern master data domains and entity types
- Execute match, merge, and deduplication logic to create golden records
- Enforce data quality rules: completeness, consistency, accuracy, uniqueness
- Manage master data hierarchies (parent-child relationships, categorization)
- Distribute master data changes to all subscribing modules via event bus
- Produce data quality scorecards and master data coverage reports

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Domain Registry	Master data domains	Define entity types: Organization, Person, Product, Location, Account, Asset
Match & Merge Engine	Deduplication	Probabilistic and deterministic matching; survivor selection rules; merge history
Data Quality Engine	DQ rules	Completeness, conformity, uniqueness, validity, accuracy rule execution
Hierarchy Manager	Relationship trees	Parent-child hierarchies with version control and effective dating
Golden Record Store	Master repository	Single authoritative record per entity with full change history
Distribution Engine	MDM syndication	Push master data changes to subscribing modules via event bus
Stewardship Workflow	Data governance	Data steward assignment, exception queues, approval for merge/overrides

Core Data Models

Model / Entity	Purpose & Key Fields
MasterEntity { id, domain, golden_record: JsonValue, source_records: Vec<SourceRecord>, quality_score }	Golden record
SourceRecord { id, entity_id, source_system, raw_data:	Source system record

JsonValue, ingested_at, match_confidence }	
DataQualityRule { id, domain, field, rule_type, expression, severity, active }	DQ rule definition
HierarchyNode { id, entity_id, parent_id, level, path, effective_date, end_date }	Hierarchy tree node
MergeDecision { id, winning_id, losing_ids, method, decided_by, decided_at, reversible }	Merge action record
MDMSyndicationEvent { entity_id, domain, change_type, before: JsonValue, after: JsonValue }	Master data change event

Key Operations & API Surface

```
ingest_source_record(record: SourceRecord) -> UmeResult<MasterEntity>
find_golden_record(domain: &str, criteria: JsonValue) -> Option<MasterEntity>
evaluate_data_quality(entity_id: &str) -> DQReport
propose_merge(entity_ids: Vec<String>) -> MergeSuggestion
execute_merge(decision: MergeDecision) -> UmeResult<MasterEntity>
update_hierarchy(node: HierarchyNode) -> UmeResult<()>
quality_scorecard(domain: &str) -> DQScorecard
```

Domain Events

Event Topic	Trigger Condition
mdm.golden_record.created	New golden record created for entity
mdm.golden_record.merged	Two or more records merged into single golden record
mdm.quality.violation	Data quality rule violation detected
mdm.syndication.dispatched	Master data change propagated to subscribing modules

Integration Points

- CRM Module — stakeholder golden records mastered in MDM
- HR Module — employee records mastered in MDM person domain
- Finance Module — account and supplier golden records from MDM
- Supply Chain Module — product and supplier data mastered in MDM
- All org modules — consume master data via MDM distribution events

Configuration

Config Key	Description / Default
mdm.match.confidence_threshold	Minimum match confidence to auto-merge — default 0.95

mdm.quality.minimum_score	Minimum acceptable DQ score before stewardship alert — default 80
mdm.syndication.async	Distribute master data changes asynchronously — default true

Module 24: Office & Facility Management

Domain: Facilities | ume.domain.facilities | Module ID: ume.module.24

Purpose & Responsibility

Enterprise office and facility management module. Manages physical workspace lifecycle, space planning, maintenance scheduling, visitor management, environmental controls, lease management, and facility cost allocation across all organizational locations.

- Maintain the authoritative facility and space registry with floor plans and capacity data
- Manage space allocation: desk booking, room reservations, hot-desking, space utilization
- Coordinate facility maintenance: preventive maintenance schedules, work orders, contractor management
- Handle visitor and access management for physical facilities
- Manage facility leases, service contracts, and related financial obligations
- Produce space utilization reports and cost-per-seat analysis

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Facility Registry	Location & space inventory	Buildings, floors, rooms, desks, capacity, accessibility metadata
Space Allocation Manager	Booking & assignment	Room booking, desk booking, space assignments per org unit
Maintenance Manager	Work orders & PPM	Preventive maintenance schedules, reactive work orders, contractor portal
Visitor Management	Access control	Visitor registration, host notification, badge management, access logs
Lease Manager	Property agreements	Lease terms, rent schedules, break options, renewal tracking
Environmental Monitor	Building systems	HVAC, energy, occupancy sensor data integration via IoT driver
Facilities Analytics	Space & cost reporting	Utilization rates, cost-per-sqft, energy consumption, occupancy trends

Core Data Models

Model / Entity	Purpose & Key Fields
Facility { id, name, address, type, total_sqft, capacity, lease_id, status }	Facility record
Space { id, facility_id, floor, name, type, capacity, amenities, bookable }	Space/room record

SpaceBooking { id, space_id, booked_by, start_at, end_at, attendees, status }	Space reservation
WorkOrder { id, facility_id, type, description, priority, assigned_to, status, completed_at }	Maintenance work order
Lease { id, facility_id, lessor, start_date, end_date, rent_cents, review_date, options }	Facility lease record
Visitor { id, name, host_id, facility_id, purpose, scheduled_at, badge_ref, departed_at }	Visitor record

Key Operations & API Surface

```
register_facility(facility: Facility) -> UmeResult<Facility>
book_space(booking: SpaceBooking) -> UmeResult<SpaceBooking>
create_work_order(wo: WorkOrder) -> UmeResult<WorkOrder>
register_visitor(visitor: Visitor) -> UmeResult<Visitor>
space_utilization_report(facility_id: &str, period: &str) -> UtilizationReport
lease_summary() -> Vec<LeaseSummary>
```

Domain Events

Event Topic	Trigger Condition
facilities.booking.created	Space booking confirmed
facilities.work_order.overdue	Maintenance work order past due date
facilities.lease.renewal_due	Facility lease renewal date approaching
facilities.occupancy.threshold_exceeded	Facility occupancy threshold exceeded

Integration Points

- HR Module — employee headcount drives space planning and desk allocation
- Schedule Management Module — room bookings appear in organizational calendar
- Finance Module — lease costs and facility expenses fed to accounts
- IT Module — network and device assets linked to physical space records
- ESG Module — energy and environmental data fed to carbon accounting

Configuration

Config Key	Description / Default
facilities.booking.advance_days	Maximum advance booking window in days — default 90
facilities.maintenance.ppm_lead_days	Days before PPM due to create work order — default 14

facilities.occupancy.alert_threshold_pct	Occupancy percentage triggering alert — default 90

Module 25: Operations Management

Domain: Operations | ume.domain.operations | Module ID: ume.module.25

Purpose & Responsibility

Enterprise operations management module providing operational planning, capacity management, service delivery tracking, SLA management, operational KPI monitoring, and continuous improvement program management across all organizational operating units.

- Maintain operational capacity models: headcount, equipment, throughput by unit
- Track service delivery performance against agreed SLAs and OLAs
- Coordinate operational planning: shift scheduling, resource allocation, workload balancing
- Manage continuous improvement programs: Lean, Six Sigma, process improvement initiatives
- Monitor and alert on operational KPI exceptions
- Produce operations dashboards and management reporting

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Capacity Manager	Resource capacity	Headcount, equipment, space capacity models per operating unit
SLA Monitor	Service performance	SLA/OLA definitions, measurement, breach detection, reporting
Operational Planner	Shift & resource planning	Schedule, resource allocation, demand-supply matching
CI Program Manager	Continuous improvement	Improvement initiatives with PDCA/DMAIC cycle tracking
Ops KPI Monitor	Performance tracking	Real-time KPI feeds with exception alerting and trend analysis
Ops Risk Tracker	Operational risk	Operational risk events, near-misses, root cause analysis
Ops Reporting	Management dashboards	Daily/weekly/monthly ops reports, trend charts, exception logs

Core Data Models

Model / Entity	Purpose & Key Fields
OperatingUnit { id, name, type, org_unit_id, capacity_model: JsonValue, sla_ids }	Operating unit record
SLADefinition { id, service, metric, target, unit, measurement_window, penalty_rule }	SLA specification

SLAMeasurement { id, sla_id, period, actual_value, target_value, compliant, notes }	SLA performance record
CIInitiative { id, title, methodology, problem_statement, owner_id, phase, savings_cents }	Continuous improvement initiative
OpsKPI { id, unit_id, metric_name, value, target, period, alert_flag }	Operational KPI data point
OpsRiskEvent { id, unit_id, description, severity, root_cause, corrective_action, status }	Operational risk event

Key Operations & API Surface

```
register_operating_unit(unit: OperatingUnit) -> UmeResult<OperatingUnit>
define_sla(sla: SLADefinition) -> UmeResult<SLADefinition>
record_sla_measurement(m: SLAMeasurement) -> UmeResult<()>
create_ci_initiative(init: CIInitiative) -> UmeResult<CIInitiative>
record_ops_kpi(kpi: OpsKPI) -> UmeResult<()>
ops_dashboard(unit_id: &str) -> OpsDashboard
```

Domain Events

Event Topic	Trigger Condition
ops.sla.breach_detected	SLA breach detected in measurement window
ops.kpi.exception	Operational KPI exceeded alert threshold
ops.ci.phase_completed	Continuous improvement initiative phase completed
ops.risk_event.logged	Operational risk event or near-miss recorded

Integration Points

- HR Module — staffing levels feed capacity models
- GRC Module — operational risk events feed GRC risk register
- Analytics Module — ops KPIs feed enterprise analytics dashboards
- Process Module — operational workflows coordinated through process module
- Finance Module — operational costs and SLA penalties tracked in finance

Configuration

Config Key	Description / Default
ops.sla.breach_escalation_minutes	Minutes past SLA breach before escalation — default 30
ops.kpi.alert_deviation_pct	KPI deviation % before exception alert — default 15

Module 26: Portal, Hub & Dashboard

Domain: Portal | ume.domain.portal | Module ID: ume.module.26

Purpose & Responsibility

Enterprise portal, employee hub, and dashboard module. Provides the primary user-facing interface for the UME OS — a unified digital workspace serving personalized dashboards, navigation to all active modules, notification center, quick-action widgets, and role-based views for employees, managers, and executives.

- Serve personalized portal homepages with role-based widget composition
- Aggregate notification streams from all modules into a unified notification center
- Provide navigation and deep-link access to all installed org module surfaces
- Render executive dashboards with KPI tiles and drill-down capability
- Support custom portal page builder with widget library
- Manage portal themes using design system tokens

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Widget Engine	Dashboard widgets	Configurable widget catalog: KPI tiles, charts, lists, actions, embeds
Page Builder	Custom portals	Drag-and-drop page composition with role-based visibility rules
Notification Center	Unified inbox	Aggregate notifications from all modules; read/dismiss/action from portal
Navigation Manager	Module links	Dynamic navigation structure reflecting installed and permitted modules
Theme Manager	Design tokens	Portal theming via design system tokens; light/dark/custom themes
Quick Actions	Action shortcuts	Configurable quick-action buttons surfacing common cross-module actions
Search Bar	Global search	Global search across knowledge, content, people, and module records

Core Data Models

Model / Entity	Purpose & Key Fields
PortalPage { id, name, audience_roles, layout, widgets: Vec<WidgetPlacement>, theme_id }	Portal page configuration
Widget { id, type, data_source, config:	Dashboard widget definition

JsonValue, permissions, refresh_sec }	
Notification { id, recipient_id, source_module, type, title, body, action_url, read_at }	Portal notification
PortalTheme { id, name, tokens: DesignTokenSnapshot, org_id, active }	Portal theme configuration
QuickAction { id, label, icon, action_type, target_module, permission_required }	Quick action button

Key Operations & API Surface

```
get_portal_page(user_id: &str, page_id: &str) -> PortalPage
get_notifications(user_id: &str, filter: NotifFilter) -> Vec<Notification>
mark_notification_read(notif_id: &str) -> UmeResult<()>
create_page(page: PortalPage) -> UmeResult<PortalPage>
register_widget(widget: Widget) -> UmeResult<Widget>
portal_search(user_id: &str, query: &str) -> Vec<SearchResult>
```

Domain Events

Event Topic	Trigger Condition
portal.notification.created	New notification pushed to portal notification center
portal.page.viewed	Portal page view recorded (for analytics)
portal.search.executed	Global search query executed

Integration Points

- Analytics Module — KPI widget data served from analytics dashboard feed
- Communications Module — in-app notifications delivered via portal notification center
- All org modules — register portal widgets and notification types during boot
- Design Module — portal themes consume design system token snapshots
- Knowledge Module — global search includes knowledge article results

Configuration

Config Key	Description / Default
portal.notifications.max_unread	Max unread notifications before badge overflow — default 99
portal.widget.default_refresh_sec	Default widget auto-refresh interval — default 300
portal.search.max_results	Maximum global search results returned — default 20

Module 27: Portfolio & Program Management

Domain: Portfolio | ume.domain.portfolio | Module ID: ume.module.27

Purpose & Responsibility

Enterprise portfolio, program, and project management module. Provides strategic portfolio governance, program coordination, project delivery management, resource allocation across the project portfolio, benefits realization tracking, and portfolio-level reporting aligned to strategic objectives.

- Maintain the enterprise portfolio of programs and projects with strategic alignment mapping
- Govern portfolio selection: business case evaluation, prioritization, and approval gates
- Coordinate program management: inter-project dependencies, shared resources, program-level risks
- Manage individual project delivery: WBS, schedule, cost, quality, risk, stakeholders
- Track benefits realization against approved business cases post-delivery
- Produce portfolio health reports, resource demand views, and strategic alignment dashboards

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Portfolio Registry	Strategic portfolio	All programs and projects with strategic pillar alignment and status
Prioritization Engine	Portfolio selection	Scoring and ranking models: strategic fit, ROI, risk, resource demand
Program Manager	Program coordination	Program scope, dependencies, shared risks, issues, and milestone tracking
Project Manager	Project delivery	WBS, Gantt schedule, budget tracking, RAID log, change control
Resource Demand Manager	Capacity planning	Cross-portfolio resource demand vs supply with conflict detection
Benefits Tracker	Realization management	Link project outputs to business case benefits; measure post-delivery realization
Portfolio Analytics	Performance reporting	Portfolio health, throughput, resource utilization, strategic delivery rate

Core Data Models

Model / Entity	Purpose & Key Fields
PortfolioItem { id, name, type, pillar_ids, status, business_case_id, priority_score }	Portfolio item
Program { id, name, portfolio_item_id, projects: Vec<String>, exec_sponsor, status, risks }	Program record

Project { id, program_id, name, type, pm_id, status, budget_cents, schedule_baseline }	Project record
WBSItem { id, project_id, parent_id, name, type, duration_days, cost_cents, resources }	WBS work package
Milestone { id, project_id, name, due_date, status, dependencies, baseline_date }	Project milestone
Benefit { id, business_case_id, description, type, target_value, measured_value, period }	Benefits realization record
RAIDEntry { id, project_id, type, description, owner_id, status, impact, probability }	RAID log entry

Key Operations & API Surface

```
register_portfolio_item(item: PortfolioItem) -> UmeResult<PortfolioItem>
create_project(project: Project) -> UmeResult<Project>
update_project_status(project_id: &str, update: ProjectStatusUpdate) -> UmeResult<()>
add_wbs_item(item: WBSItem) -> UmeResult<WBSItem>
log_raid_entry(entry: RAIDEntry) -> UmeResult<RAIDEntry>
record_benefit_realization(benefit: Benefit) -> UmeResult<Benefit>
portfolio_health_report() -> PortfolioHealthReport
```

Domain Events

Event Topic	Trigger Condition
portfolio.project.milestone_at_risk	Project milestone flagged as at-risk
portfolio.project.overbudget	Project budget exceeded approved amount
portfolio.resource.conflict_detected	Resource demand conflict detected across portfolio
portfolio.benefit.shortfall	Benefits realization below business case projection

Integration Points

- Management & Strategy Module — projects aligned to strategic objectives and OKRs
- Finance Module — project budgets and actuals tracked in finance module
- Risk Management Module — project RAID risks escalated to enterprise risk register
- HR Module — resource demand linked to workforce capacity
- Requirements Module — project requirements managed through requirements module

Configuration

Config Key	Description / Default
portfolio.project.budget_overrun_alert_pct	Budget overrun % before alert — default 10
portfolio.milestone.at_risk_days	Days past due before milestone flagged at-risk — default 7

Module 28: PR & Branding System

Domain: Brand | ume.domain.branding | Module ID: ume.module.28

Purpose & Responsibility

Enterprise public relations and brand management module. Manages media relations, press releases, crisis communications, brand identity governance, brand asset distribution, reputation monitoring, and brand performance measurement across all channels and markets.

- Maintain the media contact database and journalist/influencer relationship records
- Coordinate press release and media statement lifecycle from draft to distribution
- Manage crisis communications playbooks with real-time situation tracking
- Govern brand identity standards and approve brand usage across the organization
- Monitor brand reputation and share of voice across media and social channels
- Produce PR and brand performance reports: coverage, sentiment, reach, SOV

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Media Relations Manager	Journalist & media contacts	Media contact database, beat tracking, relationship history, embargo management
Press Release Engine	Media statement lifecycle	Draft -> legal review -> approve -> distribute press release workflow
Crisis Comm Tracker	PR crisis management	Live crisis situation board with response team, talking points, statement log
Brand Standards Manager	Brand identity governance	Brand guidelines version control, brand usage approval, deviation tracking
Reputation Monitor	Media listening	Coverage monitoring, sentiment analysis, share of voice, mention tracking
Brand Asset Distributor	Asset management	Distribute approved brand assets to partners, agencies, and internal users
PR Analytics	Coverage & impact	Coverage volume, sentiment trends, message penetration, SOV analytics

Core Data Models

Model / Entity	Purpose & Key Fields
MediaContact { id, name, outlet, beat, email, tier, last_interaction_at, relationship_status }	Media contact record
PressRelease { id, title, embargo_until, status, approved_by, wire_services, distributed_at }	Press release record

CrisisSituation { id, title, severity, status, response_team, statements: Vec<Statement> }	Crisis situation tracking
BrandUsageRequest { id, requester_id, asset_ids, purpose, approved_by, approved_at }	Brand usage approval
MediaMention { id, source, headline, sentiment, reach, published_at, media_contact_id }	Media coverage mention
BrandGuidelines { id, version, sections: JsonValue, published_at, supersedes }	Brand guidelines document

Key Operations & API Surface

```
register_media_contact(contact: MediaContact) -> UmeResult<MediaContact>
create_press_release(pr: PressRelease) -> UmeResult<PressRelease>
distribute_press_release(pr_id: &str, targets: Vec<String>) ->
UmeResult<DistributionReport>
open_crisis_situation(crisis: CrisisSituation) -> UmeResult<CrisisSituation>
approve_brand_usage(request: BrandUsageRequest) -> UmeResult<()>
reputation_summary(period: &str) -> ReputationSummary
```

Domain Events

Event Topic	Trigger Condition
branding.press_release.distributed	Press release distributed to media contacts/wire services
branding.crisis.opened	Crisis situation opened and response team notified
branding.reputation.sentiment_negative	Brand sentiment dropped below negative threshold
branding.brand_usage.violation	Unauthorized brand usage detected

Integration Points

- Communications Module — crisis communications dispatched via comms module playbooks
- Design Module — brand assets governed with design module brand asset manager
- Marketing Module — PR campaigns coordinated with marketing campaign plans
- Legal Entity Module — press releases about corporate changes reviewed with legal

Configuration

Config Key	Description / Default
branding.reputation.negative_sentiment_threshold	Sentiment score below which alert fires — default -0.3
branding.press_release.legal_review_required	Always require legal review of press releases — default true

Module 29: Process, Orchestration & Workflow

Domain: Process | ume.domain.process | Module ID: ume.module.29

Purpose & Responsibility

Enterprise business process management, orchestration, and workflow module. Models, deploys, and monitors all organizational business processes. Provides BPMN-aligned process modeling, workflow template library, automated workflow execution, process performance monitoring, and continuous process improvement analytics.

- Define and version all organizational business process models (BPMN-aligned)
- Deploy and execute automated workflow instances with full audit trail
- Manage the organizational workflow template library
- Monitor process performance: cycle time, SLA compliance, bottleneck detection
- Coordinate multi-step cross-module orchestration via kernel OrchestratorEngine
- Support human task management: assignments, escalations, SLA enforcement

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Process Library	Process model repository	Versioned BPMN-aligned process definitions with metadata
Workflow Template Library	Template catalog	Reusable workflow templates for common organizational processes
Workflow Engine	Execution runtime	Instantiate, execute, and track workflow instances step by step
Human Task Manager	Manual step handling	Assign, track, escalate, and complete human tasks within workflows
Process Monitor	Performance analytics	Cycle time, throughput, bottleneck, SLA compliance per process
Orchestration Adapter	Cross-module coordination	Wrap kernel OrchestratorEngine for multi-module process steps
Process Mining	Conformance checking	Analyze event logs to discover actual process vs designed model

Core Data Models

Model / Entity	Purpose & Key Fields
ProcessDefinition { id, name, version, bpmn_xml, owner_id, status, category }	Process model definition
WorkflowInstance { id, process_id, initiator_id, status, current_step, started_at, due_at }	Active workflow instance

WorkflowStep { id, instance_id, name, type, assignee_id, status, started_at, completed_at }	Workflow step execution
HumanTask { id, step_id, assignee_id, due_at, priority, form_schema, status, completed_by }	Human task record
ProcessMetrics { process_id, period, avg_cycle_time_min, throughput, sla_compliance_pct }	Process performance metrics
WorkflowTemplate { id, name, category, steps: JsonValue, variables: JsonValue, version }	Workflow template

Key Operations & API Surface

```
define_process(def: ProcessDefinition) -> UmeResult<ProcessDefinition>
start_workflow(process_id: &str, input: JsonValue) -> UmeResult<WorkflowInstance>
complete_human_task(task_id: &str, output: JsonValue, completed_by: &str) ->
UmeResult<()>
cancel_workflow(instance_id: &str, reason: &str) -> UmeResult<()>
process_metrics(process_id: &str, period: &str) -> ProcessMetrics
list_active_workflows(filter: WorkflowFilter) -> Vec<WorkflowInstance>
workflow_template_catalog() -> Vec<WorkflowTemplate>
```

Domain Events

Event Topic	Trigger Condition
process.workflow.started	New workflow instance started
process.workflow.completed	Workflow instance reached terminal completion state
process.human_task.overdue	Human task exceeded SLA without completion
process.bottleneck.detected	Process performance analysis identified bottleneck step
process.sla.breach	Process instance exceeded SLA threshold

Integration Points

- All org modules — initiate workflows via process module as standard integration pattern
- Administration Module — admin approval workflows defined and executed here
- HR Module — onboarding, offboarding, and leave workflows managed here
- Finance Module — AP/AR approval and payment workflows managed here
- GRC Module — compliance evidence and audit workflows managed here

Configuration

Config Key	Description / Default

process.human_task.default_sla_hours	Default human task SLA in hours — default 24
process.workflow.max_concurrent_per_process	Max concurrent instances per process definition — default 1000
process.escalation.enabled	Enable automatic escalation for overdue tasks — default true

Module 30: Product, Services & Solution Management

Domain: Product | ume.domain.product | Module ID: ume.module.30

Purpose & Responsibility

Enterprise product, service, solution, application, platform, and goods management module. Provides the master product catalog, product lifecycle management, roadmap planning, feature tracking, release management, pricing governance, and product performance analytics.

- Maintain the authoritative product/service catalog with lifecycle status and ownership
- Manage product roadmaps: themes, epics, features, and prioritization frameworks
- Coordinate product release management: release planning, change logs, launch checklists
- Govern pricing: price lists, discount rules, pricing changes with approval workflows
- Track product performance: adoption, revenue, NPS, defect rates, support ticket volume
- Manage product retirement: sunset planning, migration guidance, customer notification

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Product Catalog	Master catalog	Products, services, SKUs, bundles with full metadata and lifecycle status
Roadmap Manager	Feature planning	Themes, epics, feature backlog with prioritization (RICE, MoSCoW, etc.)
Release Manager	Delivery coordination	Release plans, change logs, launch checklists, go-no-go gates
Pricing Engine	Price governance	Price lists, discount rules, price change approval, competitive benchmarks
Product Analytics	Performance metrics	Adoption, revenue, NPS, defect rates, churn, feature utilization
Retirement Manager	End-of-life planning	Sunset roadmap, migration guides, customer comms, contract rundown

Core Data Models

Model / Entity	Purpose & Key Fields
Product { id, name, type, category, status, owner_id, launch_date, eol_date, pricing_id }	Product catalog entry
ProductFeature { id, product_id, name, epic_id, priority, status, effort_points, release_id }	Product feature record
Release { id, product_id, version, type, planned_date, status, changes: JsonValue }	Product release

PriceList { id, product_id, currency, items: Vec<PriceItem>, effective_date, end_date }	Product price list
ProductMetrics { product_id, period, revenue_cents, active_users, nps, support_tickets }	Product performance metrics
RetirementPlan { id, product_id, eol_date, migration_target_id, customer_comm_plan, status }	Product retirement plan

Key Operations & API Surface

```
register_product(p: Product) -> UmeResult<Product>
add_feature(f: ProductFeature) -> UmeResult<ProductFeature>
plan_release(r: Release) -> UmeResult<Release>
publish_price_list(pl: PriceList) -> UmeResult<PriceList>
record_product_metrics(m: ProductMetrics) -> UmeResult<()>
create_retirement_plan(plan: RetirementPlan) -> UmeResult<RetirementPlan>
product_catalog_summary() -> ProductCatalogSummary
```

Domain Events

Event Topic	Trigger Condition
product.launched	Product moved to active/launched lifecycle state
product.release.shipped	Product release delivered to production
product.nps.declined	Product NPS score declined below threshold
product.retirement.announced	Product retirement plan published to customers
product.price.changed	Price list updated — downstream notification triggered

Integration Points

- Sales Module — product catalog feeds sales quoting and CRM opportunity lines
- Finance Module — product pricing feeds revenue recognition and invoicing
- Engineering Module — product feature roadmap links to engineering initiatives
- Marketing Module — product launches integrated with marketing campaign calendar
- Supply Chain Module — goods/physical product management links to inventory and fulfillment

Configuration

Config Key	Description / Default
product.nps.alert_threshold	NPS score below which decline alert fires — default 30
product.release.go_nogo_lead_days	Days before release for go/no-go gate — default 3

Module 31: Production, Manufacturing & Fabrication Management

Domain: Production | ume.domain.production | Module ID: ume.module.31

Purpose & Responsibility

Enterprise production and manufacturing management module. Manages production planning, work order execution, bill of materials (BOM) governance, quality control, equipment maintenance (OEE), production capacity planning, and manufacturing cost tracking for goods-producing organizations.

- Maintain bills of materials (BOM) with version control and ECO management
- Plan and schedule production runs with capacity constraint checking
- Execute and track work orders through production stages
- Manage quality inspections, non-conformances, and corrective actions
- Monitor equipment OEE (availability, performance, quality)
- Track production costs: material, labor, overhead per work order

Sub-components & Feature Areas

Sub-component	Feature Area	Description
BOM Manager	Bill of materials	Versioned multi-level BOMs with ECO workflows
Production Scheduler	MPS & MRP	Master production schedule, material requirements planning, capacity checks
Work Order Engine	Production execution	Work order creation, stage routing, completion tracking, WIP management
Quality Management	QC & NCR	Inspection plans, quality checkpoints, NCRs, CAPA management
OEE Monitor	Equipment effectiveness	Availability, performance, quality metrics per machine/line
Production Costing	Cost tracking	Standard vs actual cost variance: material, labor, overhead per WO

Core Data Models

Model / Entity	Purpose & Key Fields
BOM { id, product_id, version, items: Vec<BOMItem>, status, effective_date }	Bill of materials
WorkOrder { id, product_id, bom_id, quantity, status, scheduled_start, actual_start, completed_at }	Production work order
QualityInspection { id, wo_id, checkpoint, pass_qty, fail_qty, inspector_id, inspected_at }	Quality inspection record

NonConformance { id, wo_id, defect_type, quantity, severity, capa_id, closed_at }	Non-conformance record
OEEMeasurement { machine_id, period, availability_pct, performance_pct, quality_pct, oee_pct }	OEE metric record
ProductionCost { wo_id, material_cents, labor_cents, overhead_cents, variance_cents }	Work order cost summary

Key Operations & API Surface

```
create_bom(bom: BOM) -> UmeResult<BOM>
schedule_production_run(plan: ProductionSchedule) -> UmeResult<Vec<WorkOrder>>
update_work_order_status(wo_id: &str, stage: ProductionStage) -> UmeResult<()>
record_quality_inspection(inspection: QualityInspection) -> UmeResult<()>
log_nonconformance(ncr: NonConformance) -> UmeResult<NonConformance>
record_oee(measurement: OEEMeasurement) -> UmeResult<()>
production_summary(period: &str) -> ProductionSummary
```

Domain Events

Event Topic	Trigger Condition
production.wo.completed	Work order production run completed
production.ncr.raised	Non-conformance record raised in quality inspection
production.oee.below_threshold	OEE metric dropped below configured threshold
production.bom.approved	New BOM version approved and activated

Integration Points

- Supply Chain Module — raw material requirements from MRP feed procurement
- Finance Module — production costs and variances feed cost accounting
- IT & Asset Module — manufacturing equipment tracked as CMDB CIs

Configuration

Config Key	Description / Default
production.oee.threshold_warning_pct	OEE % below which alert fires — default 65
production.quality.ncr_critical_severity	Severity level for auto-escalation of NCRs — default critical

Module 32: Requirements Management

Domain: Requirements | ume.domain.requirements | Module ID: ume.module.32

Purpose & Responsibility

Enterprise requirements management module providing structured capture, analysis, traceability, and validation of business, functional, and non-functional requirements across all organizational projects, products, and regulatory obligations. Maintains the requirements traceability matrix and supports change impact analysis.

- Capture and structure requirements: business, functional, non-functional, regulatory, technical
- Maintain bidirectional traceability: requirements to source, design, implementation, and test
- Manage requirements change control with impact analysis and approval workflows
- Validate requirements completeness, consistency, and testability
- Produce requirements traceability matrices and coverage reports
- Coordinate requirements elicitation workshops and sign-off processes

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Requirements Repository	Structured catalog	Requirements with type hierarchy, status, priority, and full metadata
Traceability Manager	RTM	Bidirectional linkage: stakeholder need -> requirement -> design -> test
Change Control Engine	Impact analysis	Change request evaluation with impact analysis on linked artifacts
Validation Engine	Quality checks	Completeness, uniqueness, testability, and consistency validation rules
Elicitation Manager	Workshop coordination	Structured elicitation sessions with stakeholder inputs and outputs
Requirements Analytics	Coverage & health	Requirements coverage, change rate, approval backlog, traceability gaps

Core Data Models

Model / Entity	Purpose & Key Fields
Requirement { id, type, title, description, priority, status, source_id, owner_id, version }	Requirement record
TraceLink { id, from_type, from_id, to_type, to_id, link_type, rationale }	Traceability link
RequirementChange { id, req_id, description, impact_analysis, status, approved_by }	Requirements change record

ElicitationSession { id, project_id, facilitator_id, stakeholders, inputs, requirements_captured }	Elicitation session record

Key Operations & API Surface

```
create_requirement(req: Requirement) -> UmeResult<Requirement>
link_requirement(link: TraceLink) -> UmeResult<()>
submit_change(change: RequirementChange) -> UmeResult<RequirementChange>
validate_requirements(project_id: &str) -> RequirementValidationReport
traceability_matrix(project_id: &str) -> TraceabilityMatrix
requirements_coverage_report(project_id: &str) -> CoverageReport
```

Domain Events

Event Topic	Trigger Condition
requirements.req.approved	Requirement formally approved by stakeholder
requirements.change.approved	Requirements change approved after impact analysis
requirements.traceability.gap_detected	Unlinked requirement detected (orphan in RTM)
requirements.validation.failed	Requirements validation identified quality issues

Integration Points

- Portfolio Module — project requirements linked to portfolio work breakdown
- Engineering Module — technical requirements link to ADRs and engineering initiatives
- GRC Module — regulatory requirements link to compliance obligations
- Process Module — approval workflows for requirement sign-off managed here
- Testing frameworks — requirements linked to test cases for coverage analysis

Configuration

Config Key	Description / Default
requirements.traceability.gap_alert	Alert when requirements exist without any trace links — default true
requirements.change.impact_threshold	Number of impacted artifacts requiring committee review — default 5

Module 33: Enterprise Risk Management

Domain: Risk | ume.domain.risk | Module ID: ume.module.33

Purpose & Responsibility

Enterprise risk management module providing the centralized risk register, risk appetite framework, risk treatment planning, key risk indicator monitoring, and risk reporting. Operates as the primary risk intelligence engine feeding GRC, Board Management, and Strategic Management modules.

- Maintain the enterprise risk register spanning strategic, operational, financial, and compliance risks
- Define and enforce the risk appetite framework with tolerance thresholds by risk category
- Coordinate risk treatment: avoid, mitigate, transfer, accept with control linkages
- Monitor key risk indicators (KRIs) in real time with automatic threshold alerting
- Conduct risk assessments: inherent risk, control effectiveness, residual risk scoring
- Produce risk reports for board, executive, and regulatory audiences

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Risk Register	Enterprise risk catalog	Risk identification, scoring, owner assignment, treatment status
Risk Appetite Framework	Tolerance thresholds	Risk appetite statements by category with KRI trigger thresholds
Treatment Planner	Risk response management	Avoidance, mitigation, transfer (insurance), acceptance plans with owners
KRI Monitor	Real-time risk indicators	KRI definitions, measurement feed, threshold alerting, trend analysis
Risk Assessment Engine	Scoring & evaluation	Inherent risk, control effectiveness, residual risk calculation
Risk Reporting	Executive dashboards	Risk heatmaps, KRI dashboards, top-N risk reports, board risk summaries

Core Data Models

Model / Entity	Purpose & Key Fields
Risk { id, category, title, inherent_score, residual_score, owner_id, treatment, review_date }	Risk register entry
RiskAppetiteStatement { id, category, tolerance_level, kri_threshold, approved_by }	Risk appetite definition
TreatmentPlan { id, risk_id, type, description, actions,	Risk treatment plan

owner_id, target_residual, due_date }	
KRI { id, risk_id, name, formula, current_value, green_threshold, amber_threshold, red_threshold }	Key risk indicator
RiskAssessment { id, risk_id, assessor_id, method, inherent, control_rating, residual, assessed_at }	Risk assessment record

Key Operations & API Surface

```
register_risk(risk: Risk) -> UmeResult<Risk>
assess_risk(assessment: RiskAssessment) -> UmeResult<RiskAssessment>
create_treatment_plan(plan: TreatmentPlan) -> UmeResult<TreatmentPlan>
define_kri(kri: KRI) -> UmeResult<KRI>
update_kri_value(kri_id: &str, value: f64) -> UmeResult<()>
risk_heatmap() -> RiskHeatmap
top_risks(n: usize) -> Vec<Risk>
```

Domain Events

Event Topic	Trigger Condition
risk.kri.red_threshold	KRI breached red (critical) threshold
risk.kri.amber_threshold	KRI entered amber (warning) zone
risk.treatment.overdue	Risk treatment action past due date
risk.assessment.completed	Risk assessment finalized and residual score updated
risk.appetite.breached	Risk exposure exceeded stated risk appetite

Integration Points

- GRC Module — risk register shared with GRC risk and control register
- Board Management Module — top risks surfaced in board packs
- Finance Module — financial risks and insurance records linked here
- Operations Module — operational risk events feed risk register

Configuration

Config Key	Description / Default
risk.scoring.matrix_size	Risk scoring matrix: 3x3 | 4x4 | 5x5 — default 5x5
risk.kri.evaluation_interval_min	KRI auto-evaluation frequency in minutes — default 60

Module 34: Sales Management

Domain: Sales | ume.domain.sales | Module ID: ume.module.34

Purpose & Responsibility

Enterprise sales management module covering the complete sales cycle from lead qualification through close, quota management, territory assignment, sales performance analytics, commissions, and sales forecasting. Deeply integrated with CRM, marketing, finance, and product modules.

- Manage the sales pipeline with stage-gated opportunity progression
- Define and track sales territories, quotas, and team assignments
- Coordinate sales forecasting: bottoms-up and pipeline-weighted models
- Track sales activity: calls, demos, proposals, negotiations per opportunity
- Calculate and manage sales commissions and incentive compensation
- Produce sales performance reports: conversion rates, pipeline velocity, win/loss analysis

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Opportunity Manager	Sales pipeline	Stage-gated opportunity tracking; probability weighting; close date management
Territory Manager	Sales territories	Geographic and account-based territory definitions and assignment rules
Quota Manager	Sales targets	Quota setting, distribution, and attainment tracking per rep/team/region
Forecast Engine	Revenue forecasting	Bottom-up and pipeline-weighted forecasting with scenario variance
Activity Tracker	Sales activity log	Calls, emails, demos, proposals logged per opportunity and rep
Commission Calculator	Incentive comp	Commission plans, accelerators, attainment bands, payout computation
Sales Analytics	Performance metrics	Win rate, ASP, pipeline velocity, quota attainment, funnel analytics

Core Data Models

Model / Entity	Purpose & Key Fields
SalesOpportunity { id, account_id, product_ids, stage, value_cents, probability, close_date, owner_id }	Sales opportunity
SalesActivity { id, opportunity_id, type, performed_by, performed_at, outcome, next_step }	Sales activity record

Quota { id, rep_id, territory_id, period, target_cents, attained_cents }	Sales quota assignment
SalesForecast { id, period, type, amount_cents, confidence_pct, created_by, created_at }	Sales forecast record
CommissionPlan { id, rep_id, structure: JsonValue, effective_date }	Commission plan
CommissionPayout { id, rep_id, period, earned_cents, paid_at, components: JsonValue }	Commission payout record

Key Operations & API Surface

```
create_opportunity(opp: SalesOpportunity) -> UmeResult<SalesOpportunity>
advance_stage(opp_id: &str, stage: SalesStage) -> UmeResult<()>
close_opportunity(opp_id: &str, outcome: CloseOutcome) -> UmeResult<()>
log_activity(activity: SalesActivity) -> UmeResult<SalesActivity>
set_quota(quota: Quota) -> UmeResult<Quota>
generate_forecast(period: &str) -> SalesForecast
compute_commissions(rep_id: &str, period: &str) -> CommissionPayout
```

Domain Events

Event Topic	Trigger Condition
sales.opportunity.closed_won	Opportunity successfully closed and won
sales.opportunity.closed_lost	Opportunity lost — reason and competitor captured
sales.quota.at_risk	Sales quota attainment tracking indicates at-risk status
sales.forecast.submitted	Sales forecast submitted for period
sales.commission.computed	Commission payout computed for rep and period

Integration Points

- CRM Module — sales opportunities linked to CRM account and stakeholder records
- Finance Module — closed opportunities trigger invoicing and revenue recognition
- Marketing Module — leads from marketing campaigns enter sales pipeline
- Product Module — product catalog feeds opportunity line items and pricing

Configuration

Config Key	Description / Default
sales.opportunity.stale_days	Days without activity before opportunity flagged stale — default 14
sales.forecast.confidence_floor_pct	Minimum confidence % for forecast inclusion — default 25

sales.commission.payout_frequency	Commission payout frequency: monthly | quarterly — default monthly

Module 35: Enterprise Schedule Management

Domain: Schedule | ume.domain.schedule | Module ID: ume.module.35

Purpose & Responsibility

Enterprise-wide schedule and calendar management module. Provides a unified organizational calendar aggregating events from all modules, manages enterprise scheduling policies, coordinates cross-module schedule conflicts, and synchronizes with external calendar systems.

- Maintain the enterprise-wide organizational calendar with multi-source event aggregation
- Manage enterprise scheduling policies: blackout periods, priority rules, booking constraints
- Detect and resolve schedule conflicts across modules and resources
- Synchronize with external calendar systems via calendar driver adapters
- Provide schedule analytics: utilization, conflict rate, meeting load distribution
- Coordinate recurring schedule templates for cyclic organizational processes

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Enterprise Calendar	Unified calendar	Aggregated view of all org events: meetings, milestones, reviews, filings
Schedule Policy Manager	Scheduling rules	Blackout dates, priority overrides, booking constraints, approval rules
Conflict Resolver	Conflict detection	Detect resource, room, and people scheduling conflicts with resolution suggestions
Calendar Sync	External integration	Bidirectional sync with Exchange, Google Calendar via ScheduleDriver
Recurring Schedule Manager	Template schedules	Define recurring process schedules: reviews, reporting cycles, audits
Schedule Analytics	Utilization insights	Meeting load, scheduling efficiency, conflict rate, calendar health

Core Data Models

Model / Entity	Purpose & Key Fields
ScheduleEvent { id, title, type, source_module, start_at, end_at, attendees, resource_ids }	Calendar event
SchedulingPolicy { id, name, rule_type, parameters: JsonValue, priority, active }	Scheduling policy rule
ScheduleConflict { id, event_ids, resource_id, detected_at, resolution, resolved_at }	Schedule conflict record

RecurringSchedule { id, template, rrule, owner_id, next_occurrence, active }	Recurring schedule template

Key Operations & API Surface

```
create_event(event: ScheduleEvent) -> UmeResult<ScheduleEvent>
check_conflicts(event: &ScheduleEvent) -> Vec<ScheduleConflict>
define_scheduling_policy(policy: SchedulingPolicy) -> UmeResult<SchedulingPolicy>
create_recurring_schedule(schedule: RecurringSchedule) -> UmeResult<RecurringSchedule>
enterprise_calendar(org_unit_id: &str, range: DateRange) -> Vec<ScheduleEvent>
schedule_utilization_report(period: &str) -> ScheduleUtilizationReport
```

Domain Events

Event Topic	Trigger Condition
schedule.event.created	New event added to enterprise calendar
schedule.conflict.detected	Scheduling conflict detected between events
schedule.recurring.fired	Recurring schedule instance created
schedule.policy.violated	Scheduling attempt violated a scheduling policy

Integration Points

- Board Management Module — board meetings synced to enterprise calendar
- HR Module — employee leave and shift schedules feed enterprise calendar
- Portfolio Module — project milestones and reviews appear in enterprise calendar
- Facilities Module — room bookings linked to calendar events
- All org modules — any event-bearing module publishes to enterprise calendar

Configuration

Config Key	Description / Default
schedule.default_timezone	Organization default timezone — default UTC
schedule.booking.min_notice_hours	Minimum advance notice for event booking — default 1
schedule.conflict.auto_resolve	Attempt automatic conflict resolution — default false

Module 36: Security, Privacy, Protection & Audit Management

Domain: Security | ume.domain.security | Module ID: ume.module.36

Purpose & Responsibility

Enterprise security, privacy, protection, and audit management module. Provides vulnerability management, security policy enforcement, privacy compliance (GDPR, CCPA), access management governance, security incident response, penetration test management, data loss prevention (DLP), and security audit coordination.

- Manage the security vulnerability register with severity scoring and patch coordination
- Enforce security policies and security baselines across the technology estate
- Govern data privacy: GDPR/CCPA obligations, consent management, DSAR processing
- Coordinate security incident response lifecycle: detect -> triage -> contain -> remediate -> review
- Track penetration test engagements, findings, and remediation status
- Integrate with RBAC engine to enforce access management across all org module boundaries

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Vulnerability Manager	Vuln registry	CVE tracking, CVSS scoring, patch coordination, risk acceptance
Security Policy Engine	Policy enforcement	Security baselines, hardening standards, policy compliance checks
Privacy Manager	Data privacy & consent	GDPR/CCPA registers, consent management, DSAR workflow, breach notification
Incident Response Engine	IR lifecycle	SIEM integration, incident classification, response playbooks, PIR reports
Pen Test Manager	Penetration testing	Engagement tracking, scope management, finding triage, remediation
Access Governance	IAM oversight	Access reviews, privileged access management, RBAC governance, certifications
DLP Engine	Data loss prevention	Data classification, exfiltration detection, policy-based content blocking

Core Data Models

Model / Entity	Purpose & Key Fields
Vulnerability { id, cve_id, cvss_score, severity, affected_ci_ids, status, patch_id, due_date }	Vulnerability record
SecurityIncident { id, title, severity, type, status,	Security incident

responders, detected_at, resolved_at }	
PrivacyRecord { id, type, data_subject_id, basis, retention_date, cross_border, status }	Privacy processing record
DSARRequest { id, subject_id, type, received_at, due_date, status, response_ref }	Data subject access request
PenTestEngagement { id, scope, tester, started_at, findings: Vec<PenTestFinding>, status }	Pen test engagement
AccessReview { id, resource_type, scope, reviewer_id, status, certifications, completed_at }	Access review campaign

Key Operations & API Surface

```
register_vulnerability(vuln: Vulnerability) -> UmeResult<Vulnerability>
open_security_incident(incident: SecurityIncident) -> UmeResult<SecurityIncident>
record_privacy_processing(record: PrivacyRecord) -> UmeResult<PrivacyRecord>
submit_dsar(request: DSARRequest) -> UmeResult<DSARRequest>
create_pen_test_engagement(eng: PenTestEngagement) -> UmeResult<PenTestEngagement>
launch_access_review(review: AccessReview) -> UmeResult<AccessReview>
security_posture_summary() -> SecurityPostureSummary
```

Domain Events

Event Topic	Trigger Condition
security.vulnerability.critical_unpatched	Critical vulnerability unpatched past SLA
security.incident.opened	Security incident opened and response team notified
security.privacy.breach_detected	Personal data breach detected — notification obligations triggered
security.dsar.overdue	DSAR response overdue (regulatory breach risk)
security.access_review.completed	Periodic access review certification completed

Integration Points

- IT & Asset Module — vulnerable CIs identified via CMDB
- GRC Module — security risks and controls feed GRC register
- Audit Module — all security events produce immutable audit records via LogAuditManager
- HR Module — access reviews linked to employee role and employment status changes
- Communications Module — security incident notifications dispatched via crisis playbooks

Configuration

Config Key	Description / Default
security.vulnerability.critical_patch_sla_days	Days to patch critical CVEs — default 7
security.incident.p1_response_time_min	P1 incident first response SLA — default 15
security.dsar.response_days	Regulatory DSAR response window — default 30
security.access_review.frequency_months	Access review campaign frequency — default 6

Module 37: Logistics, Supply Chain, Inventory & Warehouse Management

Domain: Supply Chain | ume.domain.supply_chain | Module ID: ume.module.37

Purpose & Responsibility

Integrated supply chain, procurement, logistics, inventory, and warehouse management module. Manages the end-to-end flow of goods and materials from supplier sourcing through procurement, receiving, warehousing, inventory management, and outbound fulfillment.

- Manage the supplier registry with performance scorecards and risk ratings
- Coordinate the full procurement lifecycle: requisition -> RFQ -> PO -> receiving -> invoice matching
- Track inventory levels with reorder triggers and safety stock management
- Manage warehouse operations: locations, putaway, picking, packing, shipping
- Optimize inbound and outbound logistics: carrier management, freight costing, tracking
- Produce supply chain analytics: OTIF, inventory turns, procurement savings, lead times

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Supplier Manager	Supplier registry	Supplier profiles, scorecards, contracts, approved vendor lists
Procurement Engine	P2P lifecycle	Requisition -> RFQ -> PO -> receiving -> 3-way match -> AP integration
Inventory Manager	Stock control	Item master, stock levels, reorder points, safety stock, cycle counting
Warehouse Management	WMS operations	Location management, receiving, putaway, picking, packing, shipping
Logistics Coordinator	Freight & carriers	Carrier management, freight cost optimization, shipment tracking
Demand Planner	Supply planning	Statistical demand forecasting, safety stock optimization, supply plans
SC Analytics	Performance metrics	OTIF, inventory turns, fill rate, procurement cycle time, savings

Core Data Models

Model / Entity	Purpose & Key Fields
Supplier { id, name, category, tier, performance_score, payment_terms, risk_rating, status }	Supplier master record
PurchaseOrder { id, supplier_id, lines: Vec<POLine>, total_cents, status, delivery_date }	Purchase order

InventoryItem { id, sku, name, category, uom, onhand_qty, reorder_point, safety_stock }	Inventory item master
WarehouseLocation { id, warehouse_id, zone, aisle, bin, capacity, item_id, qty }	Warehouse location record
Shipment { id, type, carrier, tracking_ref, origin, destination, status, eta }	Shipment/freight record
GoodsReceipt { id, po_id, supplier_id, lines: Vec<GRLine>, received_at, quality_status }	Goods receipt record

Key Operations & API Surface

```
register_supplier(s: Supplier) -> UmeResult<Supplier>
create_purchase_order(po: PurchaseOrder) -> UmeResult<PurchaseOrder>
receive_goods(receipt: GoodsReceipt) -> UmeResult<()>
adjust_inventory(item_id: &str, delta_qty: i32, reason: &str) -> UmeResult<()>
allocate_warehouse_location(item_id: &str, qty: i32) -> WarehouseLocation
create_shipment(shipment: Shipment) -> UmeResult<Shipment>
supply_chain_analytics(period: &str) -> SupplyChainAnalytics
```

Domain Events

Event Topic	Trigger Condition
sc.inventory.reorder_triggered	Inventory level reached reorder point
sc.po.received	Purchase order goods receipt completed
sc.shipment.delayed	Shipment tracking indicates delivery delay
sc.supplier.performance_declined	Supplier scorecard dropped below threshold
sc.inventory.stockout_risk	Projected stockout within configured horizon

Integration Points

- Finance Module — POs and GRs feed AP; inventory valuation feeds accounting
- Production Module — raw material requirements from MRP feed procurement
- ESG Module — supplier ESG scores feed supply chain sustainability reports
- MDM Module — supplier and product master data governed via MDM

Configuration

Config Key	Description / Default
sc.inventory.reorder_lead_days	Replenishment lead time for reorder calculation — default 14
sc.supplier.performance_alert_threshold	Scorecard below which alert fires — default 70

sc.po.3way_match_tolerance_pct	Tolerance for 3-way match variance — default 2

Module 38: Team & Cooperative Management

Domain: Teams | ume.domain.teams | Module ID: ume.module.38

Purpose & Responsibility

Enterprise team formation, governance, and cooperative work management module. Manages team composition, team charters, cross-functional collaboration, team health and performance measurement, and the governance of cooperative structures (working groups, guilds, communities of practice, joint ventures).

- Maintain team registry with composition, charter, and reporting structure
- Manage team formation workflows: request -> approval -> onboarding -> dissolution
- Track team health and performance: delivery, engagement, psychological safety, morale
- Govern cooperative structures: guilds, communities of practice, cross-functional task forces
- Facilitate team retrospectives and action tracking
- Produce team analytics: team size, tenure, health scores, collaboration patterns

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Team Registry	Team catalog	All active teams with composition, charter, status, and KPIs
Team Charter Manager	Charter governance	Team purpose, scope, roles, working agreements, decision rights
Health Monitor	Team health	Morale, delivery, collaboration, and psychological safety metrics
Cooperative Manager	Guilds & CoPs	Community of practice governance: membership, purpose, artifacts, events
Retro Engine	Retrospective facilitation	Structured retro templates with action item tracking and trend analysis
Collaboration Analytics	Team metrics	Cross-team collaboration patterns, dependency maps, team topology insights

Core Data Models

Model / Entity	Purpose & Key Fields
Team { id, name, type, lead_id, members: Vec<TeamMember>, status, org_unit_id, charter_id }	Team record
TeamCharter { id, team_id, purpose, scope, working_agreements, decision_rights, version }	Team charter document
TeamHealthScore { team_id, period, delivery_score,	Team health assessment

engagement_score, safety_score, overall }	
Cooperative { id, name, type, members: Vec<String>, purpose, artifacts, meeting_cadence }	Cooperative structure record
Retrospective { id, team_id, period, items: Vec<RetroItem>, actions: Vec<RetroAction> }	Team retrospective record

Key Operations & API Surface

```
register_team(team: Team) -> UmeResult<Team>
create_charter(charter: TeamCharter) -> UmeResult<TeamCharter>
record_health_score(score: TeamHealthScore) -> UmeResult<()>
create_cooperative(coop: Cooperative) -> UmeResult<Cooperative>
run_retrospective(retro: Retrospective) -> UmeResult<Retrospective>
team_analytics(team_ids: Vec<String>) -> TeamAnalytics
```

Domain Events

Event Topic	Trigger Condition
teams.team.formed	New team registered and charter approved
teams.team.dissolved	Team formally dissolved
teams.health.below_threshold	Team health score dropped below threshold
teams.retro.actions_overdue	Retrospective action items overdue without completion

Integration Points

- HR Module — team members linked to employee profiles
- Work Management Module — team work items and task boards managed here
- Portfolio Module — project teams linked to portfolio items
- Analytics Module — team performance metrics aggregated in enterprise analytics

Configuration

Config Key	Description / Default
teams.health.assessment_frequency_weeks	Team health check cadence — default 4
teams.health.alert_threshold	Overall health score below which alert fires — default 60

Module 39: Organization Templating System

Domain: Templates | ume.domain.templates | Module ID: ume.module.39

Purpose & Responsibility

Enterprise-grade templating system providing versioned document, notification, report, and data templates for all organization modules. Supports rich variable substitution, conditional logic, multi-language/locale rendering, and brand-governed output generation.

- Maintain the organizational template library with versioning and change management
- Provide template rendering engine with variable injection and conditional logic
- Manage multi-language and locale-aware template variants
- Integrate with design system for brand-governed output formatting
- Govern template lifecycle: draft -> review -> approve -> publish -> deprecate
- Support template categories: document, email, notification, report, contract, form

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Template Library	Versioned template store	All org templates with categories, variables, locale variants, versions
Template Renderer	Rendering engine	Variable substitution, conditional blocks, loops, nested templates
Locale Manager	i18n support	Per-locale template variants; locale fallback chain; translation management
Brand Integration	Design token injection	Apply design system tokens to template rendering for brand consistency
Template Lifecycle	Governance workflow	Draft -> peer review -> approval -> publish -> deprecate lifecycle
Template Analytics	Usage metrics	Template usage rates, render times, error rates, adoption by module

Core Data Models

Model / Entity	Purpose & Key Fields
Template { id, name, category, version, body, variables: Vec<TemplateVar>, locale, status }	Template definition
TemplateVar { name, type, required, default, description }	Template variable specification
TemplateRenderRequest { template_id, locale, variables: HashMap<String,JsonValue> }	Render invocation

TemplateRenderResult { request_id, output, rendered_at, duration_ms }	Rendered output
TemplateLocale { template_id, locale, body, version, translator_id }	Locale-specific template variant

Key Operations & API Surface

```
create_template(t: Template) -> UmeResult<Template>
publish_template(template_id: &str) -> UmeResult<()>
render(request: TemplateRenderRequest) -> UmeResult<TemplateRenderResult>
list_templates(category: Option<String>) -> Vec<Template>
get_template_variables(template_id: &str) -> Vec<TemplateVar>
add_locale_variant(locale: TemplateLocale) -> UmeResult<()>
```

Domain Events

Event Topic	Trigger Condition
templates.template.published	Template moved to published/active state
templates.template.deprecated	Template deprecated — consumers notified to migrate
templates.render.error	Template render failed — error captured with context

Integration Points

- Communications Module — email and notification templates rendered via template engine
- HR Module — offer letters, contracts, pay slips rendered from templates
- Finance Module — invoice, statement, and tax templates rendered here
- Design Module — brand tokens and assets injected into template renders
- All org modules — any module requiring formatted output uses template engine

Configuration

Config Key	Description / Default
templates.render.timeout_ms	Maximum render time before timeout — default 5000
templates.locale.fallback_chain	Locale fallback chain — default en-US,en
templates.cache.ttl_sec	Rendered template cache TTL — default 300

Module 40: Enterprise Work Management

Domain: Work | ume.domain.work | Module ID: ume.module.40

Purpose & Responsibility

Enterprise work management module providing unified task management, work item tracking, prioritization, workload balancing, and productivity analytics across all organizational teams and domains. Serves as the operational work layer above the Process module, translating workflows into assignable, trackable work items.

- Manage work items across all types: tasks, bugs, stories, actions, to-dos
- Coordinate kanban-style boards, sprint planning, and backlog management
- Provide workload balancing and capacity-aware assignment recommendations
- Track time spent, velocity, and throughput per person and team
- Generate productivity and work analytics: WIP, lead time, cycle time, throughput
- Support escalation and dependency management across work items

Sub-components & Feature Areas

Sub-component	Feature Area	Description
Work Item Registry	Task store	All work items with type, status, priority, assignee, and parent hierarchy
Board Manager	Kanban & sprint boards	Configurable kanban boards and sprint boards per team or project
Backlog Manager	Prioritized work queues	Prioritized backlogs with drag-drop ordering and estimation
Workload Manager	Assignment & balancing	Workload view per person; capacity-aware assignment suggestions
Time Tracker	Time recording	Time logs against work items; timesheet approval workflow
Dependency Manager	Work item links	Blocks/blocked-by, relates-to, duplicates dependency tracking
Work Analytics	Productivity metrics	WIP, lead time, cycle time, throughput, velocity, burndown charts

Core Data Models

Model / Entity	Purpose & Key Fields
WorkItem { id, type, title, status, priority, assignee_id, team_id, parent_id, due_date, estimate_h }	Work item record
Board { id, name, team_id, type, columns: Vec<BoardColumn>, filter_rules }	Work board configuration

Sprint { id, team_id, goal, items: Vec<String>, start_date, end_date, velocity_points }	Sprint record
TimeLog { id, work_item_id, worker_id, date, hours, notes, approved }	Time log entry
WorkDependency { id, from_item_id, to_item_id, type, created_at }	Work item dependency link

Key Operations & API Surface

```
create_work_item(item: WorkItem) -> UmeResult<WorkItem>
update_status(item_id: &str, status: WorkStatus) -> UmeResult<()>
create_board(board: Board) -> UmeResult<Board>
start_sprint(sprint: Sprint) -> UmeResult<Sprint>
log_time(log: TimeLog) -> UmeResult<TimeLog>
add_dependency(dep: WorkDependency) -> UmeResult<()>
work_analytics(team_id: &str, period: &str) -> WorkAnalytics
```

Domain Events

Event Topic	Trigger Condition
work.item.completed	Work item status moved to done/completed
work.item.blocked	Work item blocked by unresolved dependency
work.sprint.completed	Sprint closed; velocity recorded
work.overdue.escalated	Work item past due date without completion
work.wip.limit_exceeded	Kanban WIP limit exceeded on board column

Integration Points

- Process Module — workflow step completions create work items for human tasks
- Portfolio Module — portfolio deliverables broken into work items here
- HR Module — time logs feed timekeeping and payroll computation
- Teams Module — work items organized per team with board visibility
- Analytics Module — work metrics feed enterprise productivity analytics

Configuration

Config Key	Description / Default
work.default_sprint_length_days	Sprint cadence in days — default 14
work.wip.default_limit	Default WIP limit per board column — default 5
work.overdue.escalation_days	Days past due before auto-escalation — default 3

Module 41: Custom UME Modules

Purpose & Responsibility

The UME OS provides a first-class custom module extension framework enabling Anthropic-approved modules to be added to the kernel module registry alongside the built-in organization modules. Custom UME modules follow the full KernelModule contract and benefit from all kernel services: RBAC, event bus, supervision, storage, orchestration, and observability.

Custom UME Module Design Principles

1. Implement the KernelModule trait fully — all lifecycle methods are required.
2. Declare all module dependencies in dependencies() — the registry validates at boot.
3. Use kernel service facades only — never call other modules directly.
4. Register all event topics and API routes in describe() -> ModuleDescriptor.
5. Follow the domain event naming convention: {domain}.{resource}.{verb}.
6. Enforce RBAC on all mutating operations via the kernel RbacEngine.
7. Write audit records for all state-changing operations.
8. Custom module IDs must follow the namespace: custom.ume.{vendor}.{module_name}.

Custom Module Registration API

```
// Register a custom module during kernel boot or dynamically post-boot
kernel.registry.register_module(Arc::new(MyCustomModule::new(config)))?;

// Module descriptor for kernel introspection
ModuleDescriptor {
    id:           "custom.ume.acme.crm_extension",
    name:         "ACME CRM Extension",
    version:      "1.2.0",
    domain:       DomainArea::Custom("acme.crm_extension"),
    dependencies: vec!["org.crm", "org.analytics"],
    events:       vec!["acme.crm_ext.sync.completed"],
    permissions:  vec!["acme.crm_ext.read", "acme.crm_ext.write"],
}
```

Custom Module Scaffolding Template

UME provides a code generation scaffold (via tools/generate_custom_module.ps1 or the equivalent Rust CLI tool) that produces a fully wired custom module crate with:

- KernelModule trait implementation with all required methods
- Module-specific config struct with serde derives
- In-memory repository stub with pluggable backend trait
- RBAC permission set definition and guard helper
- Domain event publishing hooks in mutation operations
- Audit record emission stubs
- Unit test harness with contract test examples
- ModuleDescriptor with all required fields pre-filled

Module 42: Custom Organization Modules

Purpose & Responsibility

Beyond UME-ecosystem custom modules, any organization can develop and deploy proprietary organization modules specific to their industry, business model, or operational context. Custom organization modules are domain-isolated, kernel-integrated extensions that add capabilities not covered by the built-in module catalog.

Organizational Domain Examples

Industry / Context	Example Custom Organization Modules
Healthcare	Clinical record management, formulary management, patient journey orchestration
Financial Services	Trading desk module, fund administration, regulatory capital calculation
Energy	Grid asset management, energy trading, emissions credit registry
Retail	Store operations module, loyalty program, planogram management
Government / Public Sector	Permit management, benefits administration, public service request tracking
Education	Student lifecycle, academic calendar, research grant management
Real Estate	Property lifecycle, tenant management, lease accounting module
Media & Entertainment	Rights management, content monetization, audience engagement platform
Manufacturing (Extended)	Tool management, die maintenance, tooling lifecycle
Logistics Provider	Fleet management, driver compliance, last-mile optimization

Custom Organization Module Framework

Framework Component	Description
Module SDK	Rust SDK providing KernelModule scaffolding, service access helpers, and testing utilities
Module Manifest (module.toml)	Declarative configuration: ID, version, domain, dependencies, permissions, event topics
Module Store Interface	Optional: submit modules to the UME Module Store for discovery and distribution
Module Sandbox Mode	Run custom modules in isolated executor pool with restricted kernel service access for safety
Hot-Load Support	Load new module versions without kernel restart via DeviceBus hot-swap semantics

Module Versioning	Semver-governed module versions; backward compatibility enforced on kernel facade calls
Module Health Dashboard	Auto-generated health dashboard widget registered in portal on module boot

Security & Isolation Requirements

- Custom organization modules MUST declare all required permissions in module.toml before boot.
- Undeclared permission requests at runtime return UmeError::AuthorizationFailed — no implicit grants.
- Custom modules run in org.custom executor pool by default — isolated from critical kernel executors.
- All custom module domain events are namespaced under org.custom.{vendor}.* to prevent topic collisions.
- Custom modules are subject to the same audit and observability requirements as built-in modules.
- Production deployments should run custom modules through the Module Sandbox for isolation verification.

Appendix A: Integration Map — Module Cross-References

The following table summarizes the primary integration relationships between built-in organization modules. All integrations are mediated through the kernel event bus or kernel facade calls — never direct module-to-module calls.

Source Module	Integrates With (Primary)
Administration (01)	HR (16), Legal Entity (13), GRC (15), Work Management (40), Portal (26)
Analytics (02)	All modules (event subscriber), Portal (26), Strategy (21), Risk (33)
Backup/Recovery (03)	StorageManager, All modules (snapshot), Security (36), Audit
Board Management (04)	Legal Entity (13), GRC (15), Finance (14), Admin (01), Schedule (35)
Business Development (05)	Strategy (21), Finance (14), Legal Entity (13), Portfolio (27), Sales (34)
CMS (06)	Admin (01), Knowledge (19), HR (16), Legal Entity (13), Backup (03)
Communications (07)	All modules (notifications), HR (16), Security (36), Portal (26), CRM (08)
CRM (08)	Sales (34), Communications (07), Finance (14), MDM (23)
Design (09)	CMS (06), Portal (26), PR & Branding (28), Product (30), Templates (39)
Engineering (10)	IT (18), Product (30), Risk (33), Requirements (32), Analytics (02)
ESG/CSR (11)	Supply Chain (37), Finance (14), GRC (15), Analytics (02), Operations (25)
Legal Entity (13)	Board (04), Finance (14), GRC (15), Admin (01), MDM (23)
Finance (14)	Legal Entity (13), HR (16), Investment (17), Supply Chain (37), Analytics (02)
GRC (15)	Legal Entity (13), Finance (14), Security (36), Admin (01), Board (04)
HR (16)	Admin (01), Finance (14), IT (18), Learning (20), Analytics (02)
Investment (17)	Finance (14), Risk (33), Board (04), Biz Dev (05)
IT (18)	Security (36), Engineering (10), HR (16), Finance (14), GRC (15)
Knowledge (19)	CMS (06), HR (16), Analytics (02), Learning (20), Process (29)
Learning (20)	HR (16), Knowledge (19), Analytics (02)

Strategy (21)	Analytics (02), Board (04), Biz Dev (05), Finance (14), All modules (OKR cascade)
Marketing / Soko (22)	CRM (08), Sales (34), Analytics (02), Communications (07), Design (09)
MDM (23)	CRM (08), HR (16), Finance (14), Supply Chain (37), All modules (master data)
Facilities (24)	HR (16), Schedule (35), Finance (14), IT (18), ESG (11)
Operations (25)	HR (16), GRC (15), Analytics (02), Process (29), Finance (14)
Portal (26)	Analytics (02), Communications (07), All modules (widget registry), Design (09)
Portfolio (27)	Strategy (21), Finance (14), Risk (33), HR (16), Requirements (32)
PR & Branding (28)	Communications (07), Design (09), Marketing (22), Legal Entity (13)
Process (29)	All modules (workflow initiation), Admin (01), HR (16), Finance (14), GRC (15)
Product (30)	Sales (34), Finance (14), Engineering (10), Marketing (22), Supply Chain (37)
Production (31)	Supply Chain (37), Finance (14), IT (18)
Requirements (32)	Portfolio (27), Engineering (10), GRC (15), Process (29)
Risk (33)	GRC (15), Board (04), Finance (14), Operations (25)
Sales (34)	CRM (08), Finance (14), Marketing (22), Product (30)
Schedule (35)	Board (04), HR (16), Portfolio (27), Facilities (24), All event-bearing modules
Security (36)	IT (18), GRC (15), Audit, HR (16), Communications (07)
Supply Chain (37)	Finance (14), Production (31), ESG (11), MDM (23)
Teams (38)	HR (16), Work (40), Portfolio (27), Analytics (02)
Templates (39)	Communications (07), HR (16), Finance (14), Design (09), All modules
Work Management (40)	Process (29), Portfolio (27), HR (16), Teams (38), Analytics (02)

Appendix B: Kernel Event Topic Registry

All kernel and organization module events follow the normative naming convention: {domain}.{resource}.{verb}. This appendix provides the complete baseline event topic registry.

Event Topic Pattern	Owning Module
kernel.boot, kernel.bootstrap.completed	Kernel Core
kernel.module.registered, kernel.module.state_changed	Module Registry
security.rbac.denied, security.rbac.allowed	RBAC Engine
audit.record.created	Log/Audit Manager
admin.*	Administration Module
analytics.*	Analytics Module
backup.*	Backup & Recovery Module
board.*	Board Management Module
bizdev.*	Business Development Module
cms.*	Content Management Module
comms.*	Communications Module
crm.*	CRM Module
design.*	Design Module
eng.*	Engineering Module
esg.*	ESG/CSR Module
chombo.*	Legal Entity Module (Chombo)
finance.*	Finance & Accounting Module
grc.*	GRC Module
hr.*	HR Module
investment.*	Investment Management Module
it.*	IT & Asset Management Module
knowledge.*	Knowledge Management Module
learning.*	Learning Management Module
strategy.*	Management & Strategy Module
soko.*	Marketing Module (Soko)
mdm.*	MDM Module
facilities.*	Facilities Module
ops.*	Operations Management Module
portal.*	Portal Module
portfolio.*	Portfolio Management Module
branding.*	PR & Branding Module
process.*	Process & Workflow Module
product.*	Product Management Module
production.*	Production Management Module

requirements.*	Requirements Management Module
risk.*	Risk Management Module
sales.*	Sales Management Module
schedule.*	Schedule Management Module
security.*	Security & Privacy Module
sc.*	Supply Chain Module
teams.*	Teams Management Module
templates.*	Templating System Module
work.*	Work Management Module
custom.ume.*	Custom UME Modules (namespaced by vendor)
org.custom.*	Custom Organization Modules

Appendix C: RBAC Permission Namespace

All permissions follow the pattern: {domain}.{resource}.{action}. Modules declare their required permissions in their ModuleDescriptor. The RBAC engine enforces these at all kernel facade boundaries.

Permission Pattern	Domain Coverage
admin.org_unit.{read|write|delete}	Organization unit management
admin.policy.{read|write|publish}	Policy lifecycle management
analytics.metric.{read|write}	KPI and metric data access
analytics.report.{read|run}	Report generation and access
backup.job.{read|write|execute}	Backup job management
backup.restore.execute	Restore operation initiation
board.meeting.{read|write}	Board meeting management
board.resolution.{read|write}	Resolution recording
chombo.entity.{read|write|evaluate}	Legal entity operations
finance.journal.{read|write}	General ledger posting
finance.invoice.{read|write|approve}	AP/AR management
grc.risk.{read|write|evaluate}	Risk register management
grc.audit.{read|write}	Audit engagement management
hr.employee.{read|write}	Employee record management
hr.payroll.{read|run}	Payroll computation
it.incident.{read|write|resolve}	IT incident management
it.change.{read|write|approve}	Change request management

knowledge.article.{read|write|publish}	Knowledge article management
mdm.entity.{read|write|merge}	Master data management
risk.register.{read|write}	Risk register management
sales.opportunity.{read|write}	Sales pipeline management
security.incident.{read|write|respond}	Security incident response
security.vulnerability.{read|write}	Vulnerability management
soko.campaign.{read|write|evaluate}	Marketing campaign management
soko.signal.ingest	Market signal ingestion
supply_chain.po.{read|write|approve}	Purchase order management
work.item.{read|write}	Work item management
custom.{vendor}.{resource}.{action}	Custom module permissions (namespaced)

Appendix D: Build & Operations Reference

Command	Purpose
cargo check --workspace	Validate all crates compile without producing binaries
cargo build --workspace	Full workspace build
cargo build --release --workspace	Production release build
cargo run -p ume_runtime -- shell	Launch interactive CLI/REPL
cargo run -p ume_server -- --config config/ume_server.toml	Start HTTP server
cargo run -p ume_server -- health --base-url http://127.0.0.1:8080	Check server health
cargo run -p ume_server -- metrics --base-url http://127.0.0.1:8080	View server metrics
cargo run -p ume_server -- lifecycle --base-url http://127.0.0.1:8080	View lifecycle state
curl -X POST http://127.0.0.1:8080/api/v1/server/cycle	Restart server gracefully
bazel build //clients/java:ume_java_clients	Build all Java clients
bazel run //clients/java:impande_client -- --base-url http://127.0.0.1:8080 --gui	Launch HRIS client GUI
bazel run //clients/java:soko_client -- --base-url http://127.0.0.1:8080 --shell	Marketing client CLI
cd clients/java && mvn -q test	Run Java client test suite
cargo clean && cargo check --workspace	Clean rebuild from scratch